

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	1
1.1 Overview of VLSI Testing	1
1.1.1 Definition and Basics of VLSI Testing	1
1.1.2 Types of Faults in VLSI Circuits	1
1.1.3 Importance of Testing in Modern Digital Systems	2
1.2 Built-In Self-Test (BIST)	2
1.2.1 Definition and Architecture of BIST	2
1.2.2 Advantages of BIST over External Testing	3
1.2.3 Role of TPG in BIST	3
1.3 Test Pattern Generation in Digital Circuits	4
1.3.1 Deterministic Testing	4
1.3.2 Exhaustive Testing	4
1.3.3 Pseudo-Random Testing	4
1.4 Need for Low Power Testing	5
1.4.1 Power Dissipation During Test Mode	5
1.4.2 Dynamic Power Equation	5
1.4.3 Consequences of High Switching Activity	5
1.5 Applications of Low Power Testing	6
CHAPTER 2 PROBLEM DEFINITION AND OBJECTIVES	7
2.1 Problem Statement	7
2.2 Limitations of Conventional LFSR-Based Testing	7
2.2.1 High Switching Activity	8
2.2.2 Multiple Bit Transitions	8
2.2.3 Reliability Issues	8
2.3 Objectives of the Proposed System	9
CHAPTER 3 LITERATURE REVIEW	10
3.1 Overview of Low Power Testing Techniques	10
3.2 Review of Existing LFSR-Based Methods	10
3.2.1 Conventional LFSR Testing	10

3.2.2 Modified LFSR Approaches	11
3.2.3 Gray Code Based Techniques	11
3.3 Comparative Summary of Research Work	12
3.4 Research Gap and Motivation	13
CHAPTER 4 METHODOLOGY	14
4.1 Overview of Methodology	14
4.2 Methodology Flow	14
4.3 Methodology Flow Diagram	16
4.4 Design Strategy	17
CHAPTER 5 FUNDAMENTAL CONCEPTS	18
5.1 Test Pattern Generator (TPG)	18
5.1.1 Definition and Purpose	18
5.1.2 Role of TPG in BIST	18
5.2 Linear Feedback Shift Register (LFSR)	19
5.2.1 Structure of LFSR	19
5.2.2 Working Principle	20
5.2.3 Feedback Polynomial	20
5.2.4 Limitations of Conventional LFSR	21
5.3 Gray Code and Single-Input-Change Sequences	21
5.3.1 Gray Code Conversion	22
5.3.2 Why SIC Sequences Reduce Power	22
5.4 Braun Array Multiplier as Circuit Under Test	23
5.4.1 Structure of Braun Array Multiplier	24
5.4.2 Role as CUT	24
CHAPTER 6 PROPOSED LP-TPG ARCHITECTURE	25
6.1 Overview of Proposed System	25
6.2 LP-LFSR Design and Algorithm	25
6.2.1 LP-LFSR Feedback Polynomial	25
6.2.2 Seed Initialization	26
6.2.3 Clock Enable Mechanism	27
6.2.4 LP-LFSR Algorithm Steps	27

6.3 Architecture Components	28
6.3.1 LP-LFSR Module	28
6.3.2 M-Bit Counter	28
6.3.3 Gray Code Generator	29
6.3.4 XOR Array	29
6.4 Block Diagram	30
6.5 System Flow Chart	31
CHAPTER 7 TOOLS AND TECHNOLOGIES	32
7.1 Target FPGA Platform — Artix-7 (xc7a35tcbg236-1)	32
7.2 Verilog HDL	33
7.3 Xilinx Vivado 2025.1	34
7.3.1 Project Creation	35
7.3.2 Design Execution Flow	36
7.4 Testbench Design	37
7.5 Complete Design Flow	37
CHAPTER 8 RTL DESIGN AND IMPLEMENTATION	38
8.1 Module Hierarchy	38
8.2 Verilog Module Design	39
8.2.1 LP-LFSR Module (lfsr1 / lfsr01)	39
8.2.2 Counter Module (count1 / count01)	39
8.2.3 Gray Code Generator Module (gray1 / gray01)	39
8.2.4 TPG Integration Module (tpg01 / tpg02)	40
8.2.5 Top-Level Module (tpgmain8)	40
8.2.6 Braun Array Multiplier Module	40
8.3 RTL Schematic	41
8.4 Synthesis Schematic	43
8.5 Synthesis Results and Resource Utilization	44
CHAPTER 9 PROJECT CODE	45
CHAPTER 10 SIMULATION RESULTS	66
10.1 Testbench Configuration	66
10.2 LP-LFSR TPG Simulation Waveform	67
10.3 Conventional LFSR TPG Simulation Waveform	68

10.4 Simulation Verification and Observations	69
10.5 Test Case Verification Table	70
CHAPTER 11 POWER ANALYSIS RESULTS	71
11.1 Power Analysis Methodology	71
11.2 LP-LFSR TPG Power Report	72
11.3 Conventional LFSR TPG Power Report	74
11.4 Comparative Power Analysis	76
11.5 Comparative Resource Utilization	77
11.6 COMPARISON: EXISTING PROJECT vs PROPOSED PROJECT	77
11.6.1 Architecture Comparison	78
11.6.2 Timing Results: Existing vs Proposed	79
11.6.3 Simulation and Power Results Comparison	80
CHAPTER 12 ADVANTAGES AND DISADVANTAGES	81
12.1 Advantages	81
12.2 Disadvantages	82
CHAPTER 13 APPLICATIONS	83
CHAPTER 14 CONCLUSION	84
CHAPTER 15 FUTURE SCOPE	85
CHAPTER 16 REFERENCES	86

LIST OF FIGURES

Figure 1:BIST Block diagram	2
Figure 2:BIST basic flow	3
Figure 3:TPG basic flow	4
Figure 4:Methodology flow chart	16
Figure 5:Example of LFSR	19
Figure 6:Basic example of gray code convertor	21
Figure 7:Basic Braun array multiplier	23
Figure 8:LOW POWER TPG BLOCK DIAGRAM	30
Figure 9:LOW POWER TPG ALGORITHM FLOWCHART	31
Figure 10:Xilinx Artix-7 FPGA Board	32
Figure 11:Basic RTL Schematic diagram	41
Figure 12:Expanded RTL Schematic diagram	42
Figure 13:Synthesis Schematic diagram	43
Figure 14:LP-TPG simulation waveform	67
Figure 15:Conventional TPG simulation waveform	68
Figure 16:LP-TPG Power Report (Pre -saif)	72
Figure 17:LP-TPG Power Report(post-SAIF)	73
Figure 18:Conventional TPG Power Report (pre-saif)	74
Figure 19:Conventional TPG Power Report(post-saif)	75
Figure 20:Existing Design - LOW POWER LFSR-Based TPG	78
Figure 21: Proposed Design — LP-TPG (Our Implementation, Artix-7 / Vivado 2025.1)	78
Figure 22:Existing Design (Gated Clock) — TIMING FAILED	79
Figure 23:Proposed Design (Synchronous Clock Enable) — TIMING PASSED	79

LIST OF TABLES

Table 1:Example of Multiple Bit Transitions in Conventional LFSR	8
Table 2:Comparative Summary of Research Work on Low Power Testing	12
Table 3:TPGs seed	26
Table 4:Artix-7 (xc7a35tcp236-1) Device Specifications	33
Table 5:Verilog Modules and File Names	34
Table 6:Project Configuration	36
Table 7:Testbench Configuration Parameters	37
Table 8:Design flow summary	37
Table 9:LP-TPG Module Hierarchy	38
Table 10:LP-TPG Resource Utilization on Artix-7 (xc7a35tcp236-1)	44
Table 11:LP-TPG Hierarchical Resource Utilization	44
Table 12:Simulation Testbench Configuration	66
Table 13:LP-TPG Selected Test Case Verification	70
Table 14:Conventional TPG Selected Test Case Verification	70
Table 15:LP-LFSR TPG Synthesized Design Power Report (Pre- SAIF)	72
Table 16:LP-LFSR TPG Implemented Design Power Report (Post-SAIF)	73
Table 17:Conventional LFSR TPG Synthesized Design Power Report	74
Table 18:Conventional LFSR TPG Implemented Design Power Report (Post-SAIF)	75
Table 19:Comparative Power Analysis — LP-LFSR TPG vs Conventional LFSR TPG	76
Table 20:Comparative Utilization - LP-LFSR TPG vs Conventional LFSR TPG	77
Table 21:Architecture Differences — Existing vs Proposed	79
Table 22:Timing Analysis Comparison — Existing vs Proposed	80

CHAPTER 1

INTRODUCTION

1.1 Overview of VLSI Testing

Very Large-Scale Integration (VLSI) technology enables the integration of millions of transistors on a single semiconductor chip, allowing the design of highly complex digital systems such as microprocessors, digital signal processors, and system-on-chip (SoC) devices. As the complexity of integrated circuits continues to increase, ensuring their functional correctness and reliability becomes a critical challenge in modern electronics. Testing is therefore an essential step in the design and manufacturing process of VLSI circuits.

1.1.1 Definition and Basics of VLSI Testing

VLSI testing is the process of applying a set of input patterns to a digital circuit and observing the corresponding output responses in order to verify correct operation and identify the presence of faults. Faults in VLSI circuits may arise due to manufacturing defects, design errors, material imperfections, or environmental factors such as radiation and temperature variations. If these faults remain undetected, they can lead to incorrect system behavior and reduced reliability of electronic devices in the field.

To improve testability, Design for Testability (DFT) techniques are widely adopted in modern VLSI systems. These techniques incorporate additional hardware or design constraints during the design phase to make the circuit easier to test. One of the most effective DFT techniques is Built-In Self-Test (BIST), which integrates the testing mechanism directly within the chip itself.

1.1.2 Types of Faults in VLSI Circuits

Faults in VLSI circuits are broadly classified into the following categories:

- **Stuck-at faults:** A signal line is permanently stuck at logic 0 (stuck-at-0) or logic 1 (stuck-at-1) regardless of the applied input. Stuck-at faults are the most commonly modeled fault type in digital testing.
- **Bridging faults:** Two or more signal lines are unintentionally connected together due to manufacturing defects, causing incorrect logic behavior.
- **Delay faults:** The circuit produces correct logic values but fails to meet timing requirements due to excessive propagation delay caused by process variations or aging.
- **Transition faults:** A signal fails to make a 0-to-1 or 1-to-0 transition within the required time, typically caused by increased resistance in interconnects.

- **Short and open faults:** Unintended short circuits between nodes or open circuits in interconnections that disrupt normal signal flow.

1.1.3 Importance of Testing in Modern Digital Systems

As VLSI circuits become increasingly complex and are deployed in safety-critical applications such as automotive electronics, medical devices, aerospace systems, and communication infrastructure, the importance of thorough testing cannot be overstated. A single undetected fault in a safety-critical system can lead to catastrophic consequences. Moreover, manufacturing yield improvement, cost reduction, and product reliability all depend directly on the effectiveness of the testing process. Therefore, efficient and comprehensive testing of VLSI circuits is not optional but essential in modern digital system design.

1.2 Built-In Self-Test (BIST)

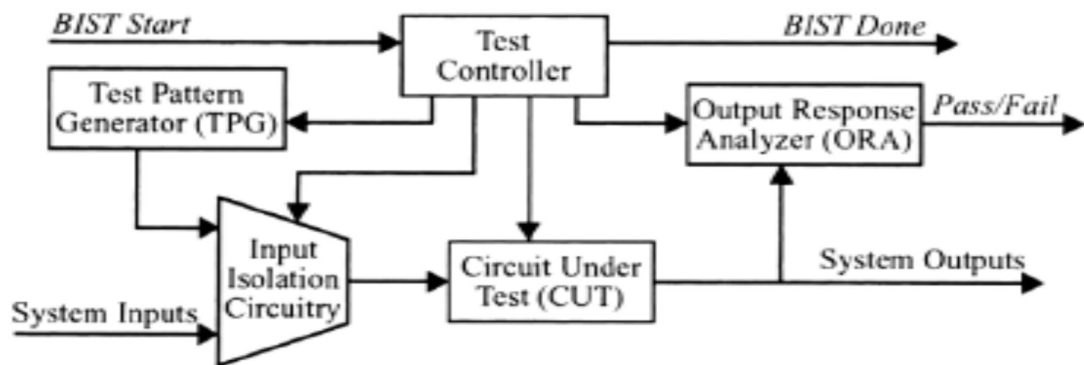


Figure 1: BIST Block diagram

Built-In Self-Test (BIST) is a Design for Testability technique in which the testing mechanism is integrated directly within the hardware of a digital system. Instead of relying solely on external testing equipment, BIST allows the circuit to generate its own test patterns and analyze the output responses internally. This approach significantly reduces the dependency on expensive external Automatic Test Equipment (ATE) and enables efficient testing of complex VLSI circuits throughout their operational lifetime.

1.2.1 Definition and Architecture of BIST

A typical BIST architecture consists of three major components working together:

- **Test Pattern Generator (TPG):** Generates the sequence of input test vectors that are applied to the Circuit Under Test. The TPG is typically implemented using a Linear Feedback Shift Register due to its ability to produce pseudo-random sequences with simple hardware.
- **Circuit Under Test (CUT):** The functional digital circuit being tested. The CUT receives the test patterns from the TPG, processes them, and produces output responses.

- **Output Response Analyzer (ORA):** Captures the output responses produced by the CUT and compares them against expected results or compresses them into a compact signature for fault detection.



Figure 2: BIST basic flow

The three components are integrated on the same chip, allowing the circuit to test itself autonomously during startup, idle periods, or dedicated test modes without requiring external intervention.

1.2.2 Advantages of BIST over External Testing

BIST offers several significant advantages compared to conventional external testing methods:

- **Reduced testing cost:** By integrating the test hardware on-chip, the dependency on expensive external ATE is significantly reduced, lowering the overall cost of testing.
- **At-speed testing:** BIST allows the circuit to be tested at its actual operating frequency, enabling detection of timing-related faults that may not be detectable at lower external test speeds.
- **Field testing capability:** BIST enables circuits to perform self-diagnostic tests during operation, allowing detection of faults that develop after deployment due to aging or environmental stress.
- **Improved fault coverage:** The pseudo-random patterns generated by LFSR-based TPGs can achieve high fault coverage for many types of faults in large digital circuits.
- **Scalability:** BIST scales well with increasing circuit complexity and is suitable for large System-on-Chip designs where traditional external testing becomes impractical.

1.2.3 Role of TPG in BIST

The Test Pattern Generator is the most critical component of any BIST architecture. The quality, diversity, and switching characteristics of the generated test patterns directly determine the fault coverage and power consumption of the testing process. An ideal TPG should generate patterns that activate all logic paths in the circuit, maintain high fault coverage, and minimize unnecessary switching transitions between consecutive test vectors. The design of an efficient TPG is therefore a fundamental challenge in BIST-based VLSI testing.

1.3 Test Pattern Generation in Digital Circuits

Test pattern generation involves producing a sequence of input vectors that, when applied to a digital circuit, stimulate its internal nodes sufficiently to detect the presence of faults. Different methods of test pattern generation offer different trade-offs between fault coverage, hardware complexity, and power consumption.

1.3.1 Deterministic Testing

Deterministic testing uses carefully designed input vectors that are specifically targeted to detect known faults in the circuit. These patterns are typically generated using Automatic Test Pattern Generation (ATPG) algorithms that analyze the circuit structure and identify the minimum set of patterns required to detect each fault. While deterministic testing provides high and predictable fault coverage, it requires extensive pre-computation and significant storage for the generated pattern sets, making it less suitable for on-chip BIST implementations.

1.3.2 Exhaustive Testing

Exhaustive testing applies all possible input combinations to the circuit, guaranteeing 100% fault coverage for any combinational circuit. For a circuit with n primary inputs, exhaustive testing requires 2^n test patterns. While this approach is theoretically complete, it becomes entirely impractical for large VLSI circuits.

1.3.3 Pseudo-Random Testing

Pseudo-random test pattern generation is widely used in BIST systems because it provides good fault coverage with simple and compact hardware implementation. Linear Feedback Shift Registers are the most commonly used circuits for generating pseudo-random patterns. An LFSR generates sequences of binary patterns that appear statistically random but are produced deterministically using shift registers and XOR feedback logic. The pseudo-random nature of the patterns allows different input combinations to be applied to the CUT, achieving good fault coverage without requiring the storage of large pattern sets. However, pseudo-random patterns generated by conventional LFSRs often cause multiple bit transitions between consecutive test vectors, leading to high switching activity and increased dynamic power consumption during testing. This limitation motivates the development of low-power test pattern generation techniques.

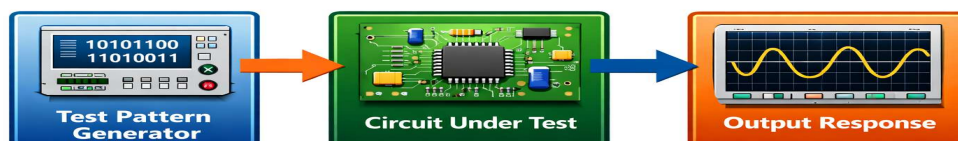


Figure 3:TPG basic flow

1.4 Need for Low Power Testing

With the rapid advancement of VLSI technology, modern integrated circuits contain millions of transistors operating at increasingly high speeds. During normal functional operation, only a portion of the circuit is active at any given time. However, during testing, especially in BIST environments, a large number of internal nodes switch simultaneously due to the application of test patterns designed to activate all circuit paths. This results in switching activity during test mode that can be several times higher than during normal operation.

1.4.1 Power Dissipation During Test Mode

The increased switching activity during test mode leads to higher dynamic power consumption. In CMOS digital circuits, dynamic power dissipation is the dominant component of total power consumption and occurs due to the charging and discharging of capacitive nodes as signals transition between logic levels. When many signals toggle simultaneously in response to test patterns, excessive power consumption occurs, which can lead to overheating, voltage drops across power supply lines, and potential permanent damage to the circuit under test.

1.4.2 Dynamic Power Equation

The dynamic power dissipation in CMOS circuits is governed by the following equation:

$$P = \frac{1}{2} CV_{dd}^2 f \alpha$$

Where:

- P — dynamic power dissipation
- α — switching activity factor (average number of transitions per clock cycle)
- C — load capacitance
- V_{dd} — supply voltage
- f — clock frequency

From this equation, it is clear that reducing the switching activity factor α directly reduces dynamic power consumption. Since supply voltage, load capacitance, and clock frequency are determined by the circuit design and operating conditions, switching activity is the most practical parameter to control during test pattern generation. This forms the fundamental motivation for the proposed Low Power Test Pattern Generator (LP-TPG) architecture.

1.4.3 Consequences of High Switching Activity

High switching activity during testing causes several serious problems in VLSI circuits:

- **Overheating:** Excessive power dissipation raises the junction temperature of the chip beyond safe operating limits, potentially causing permanent thermal damage.
- **IR drop:** Large simultaneous switching currents cause voltage drops across the power supply network, leading to incorrect logic levels and unreliable test results.
- **Electromigration:** Repeated high current densities accelerate electromigration in metal interconnects, reducing the long-term reliability of the circuit.
- **Incorrect test results:** Voltage drops and noise caused by high switching activity may trigger false fault detections or mask actual faults, reducing the accuracy of the testing process.
- **Reliability degradation:** Repeated high-power test sessions accelerate the aging of the circuit, reducing its operational lifespan.

1.5 Applications of Low Power Testing

Low power testing techniques are essential across a wide range of modern electronic applications:

- **System-on-Chip (SoC) designs:** Large SoC devices integrating multiple functional blocks require efficient low-power BIST to test all subsystems without causing thermal damage or voltage instability.
- **Mobile and portable devices:** Smartphones, tablets, and wearable electronics operate on limited battery capacity, making power-efficient testing essential during manufacturing and field diagnostics.
- **Automotive electronics:** Safety-critical automotive systems such as engine control units, airbag controllers, and advanced driver assistance systems require reliable and power-aware testing to ensure safe operation.
- **Medical devices:** Implantable and portable medical devices such as pacemakers and hearing aids operate on very limited power budgets, requiring extremely low-power testing techniques.
- **Aerospace and defence systems:** High-reliability systems operating in harsh environments require regular self-testing with minimal power impact to avoid interference with other onboard systems.
- **FPGA-based embedded systems:** Reconfigurable hardware platforms used in industrial and communication applications benefit from low-power BIST to enable in-field testing without disrupting normal operation.

CHAPTER 2

PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Statement

Testing is a critical phase in the design and manufacturing of VLSI circuits. As the complexity of integrated circuits continues to increase with advancing semiconductor technology, efficient and reliable testing techniques are essential to detect faults and ensure the correct functionality of digital systems. Built-In Self-Test (BIST) is widely adopted in modern digital systems to perform testing internally by generating test patterns and analyzing output responses without dependency on external testing equipment.

Linear Feedback Shift Registers are commonly used as Test Pattern Generators in BIST architectures because of their simple hardware implementation and ability to generate pseudo-random test sequences. However, conventional LFSR-based test pattern generators produce patterns in which multiple bits change simultaneously between consecutive test vectors. For example, consecutive outputs such as 00010011 $\rightarrow$ 11100101 involve six simultaneous bit transitions. These multiple transitions result in high switching activity at the inputs and internal nodes of the Circuit Under Test.

The excessive switching activity during testing leads to significantly increased dynamic power consumption compared to normal circuit operation. High power consumption during test mode may cause overheating of the chip, reliability degradation due to electromigration, voltage drops across the power supply network, and inaccurate test results caused by noise and signal integrity issues.

Therefore, there is a clear need for an efficient test pattern generation technique that reduces switching activity between consecutive test vectors while maintaining effective fault detection capability. The proposed Low Power Test Pattern Generator (LP-TPG) based on a modified Low Power Linear Feedback Shift Register (LP-LFSR) architecture addresses this problem by combining pseudo-random sequences with Gray code based single-input-change patterns to generate test vectors with significantly reduced switching transitions.

2.2 Limitations of Conventional LFSR-Based Testing

Linear Feedback Shift Registers are widely used in BIST architectures as Test Pattern Generators due to their simple hardware structure and efficient pseudo-random sequence generation. Despite these advantages, conventional LFSR-based testing has several important limitations particularly in terms of power consumption and switching activity during test mode.

2.2.1 High Switching Activity

The patterns generated by a conventional LFSR are pseudo-random in nature, meaning that multiple bits may change simultaneously between successive output vectors. Since there is no control over the correlation between consecutive patterns, the number of bit transitions per clock cycle can be as high as the full register width. In an 8-bit LFSR for example, up to 8 bits may toggle simultaneously between successive patterns. This results in a large number of simultaneous signal transitions at the inputs of the Circuit Under Test, causing extremely high switching activity throughout the circuit.

2.2.2 Multiple Bit Transitions

The following table illustrates the multiple bit transitions that occur in a conventional 8-bit LFSR using the maximal-length feedback polynomial $x^8+x^6+x^5+x^4+1$ with seed 10101010:

Table 1: Example of Multiple Bit Transitions in Conventional LFSR

Clock Cycle	LFSR Output	Transitions from Previous
1(seed)	10101010	—
2	01010101	8 bits change
3	10101010	8 bits change
4	11010100	3 bits change
5	01101010	4 bits change
6	00110101	5 bits change

As seen from the table, consecutive LFSR outputs regularly produce multiple simultaneous bit transitions, directly increasing switching activity in the Circuit Under Test.

2.2.3 Reliability Issues

High switching activity during testing causes several reliability problems. Excessive dynamic power dissipation raises the junction temperature of the chip, potentially beyond the safe operating limit of the device. Simultaneously, large switching currents cause IR drops across the power distribution network, which can result in incorrect logic levels being interpreted by the circuit. In severe cases, repeated high-power test sessions can accelerate electromigration in metal interconnects, permanently reducing the reliability and lifespan of the circuit. These effects may also cause incorrect test results, where actual faults are masked or false faults are detected due to noise induced by excessive switching.

2.3 Objectives of the Proposed System

The main objective of this work is to design and implement a Low Power Test Pattern Generator (LP-TPG) that reduces switching activity during the testing of VLSI circuits while maintaining effective fault detection capability. The proposed system addresses the limitations of conventional LFSR-based test pattern generators by combining pseudo-random pattern generation with Gray code based single-input-change sequences.

The specific objectives of the proposed system are as follows:

1. To design a Low Power Linear Feedback Shift Register (LP-LFSR) that generates pseudo-random seed values with a controlled clock enable mechanism to reduce unnecessary switching transitions.
2. To implement an 8-bit binary counter and Gray code generator to produce single-input-change sequences in which only one bit changes between consecutive values.
3. To combine the LP-LFSR outputs with Gray code sequences through an XOR array to generate final low-transition test patterns.
4. To implement the complete LP-TPG architecture using Verilog Hardware Description Language and verify functional correctness through behavioral simulation in Xilinx Vivado 2025.1.
5. To validate the proposed design by applying the generated test patterns to an 8×8 Braun Array Multiplier as the Circuit Under Test.
6. To analyze and compare the dynamic power consumption of the proposed LP-LFSR based TPG against a conventional LFSR based TPG using post-implementation power analysis in Xilinx Vivado 2025.1.
7. To evaluate the resource utilization of both designs on the Artix-7 FPGA device (xc7a35tcpg236-1) and demonstrate that the LP-TPG architecture achieves power reduction with acceptable hardware overhead.

CHAPTER 3

LITERATURE REVIEW

3.1 Overview of Low Power Testing Techniques

With the continuous growth in the complexity of VLSI circuits, power consumption during testing has become a major concern in modern digital system design. During test mode, the switching activity of internal nodes is significantly higher than during normal functional operation because test patterns are designed to activate a large number of circuit paths simultaneously in order to detect faults. This increased switching activity leads to excessive dynamic power consumption, which may cause overheating, voltage fluctuations, and reliability issues in the circuit under test.

Balwinder Singh, Arun Khosla, and Sukhleen Bindra identified power optimization of the Linear Feedback Shift Register as a critical requirement for low power BIST applications. Their work presented at the IEEE International Advance Computing Conference 2009 demonstrated that the LFSR, while simple and widely used, consumes unnecessary power due to uncontrolled switching transitions, and proposed optimization techniques to reduce this power overhead in BIST-based testing systems.

Zorian proposed a distributed BIST control scheme for complex VLSI devices, establishing the foundational architecture for integrating BIST components within large digital systems. His work demonstrated that distributed BIST control enables efficient testing of multiple functional blocks while managing the power and timing constraints of the overall system.

3.2 Review of Existing LFSR-Based Methods

3.2.1 Conventional LFSR Testing

Girard surveyed low-power testing techniques for VLSI circuits, identifying four main causes of excessive power during test mode: high switching activity from test patterns, parallel core activation, DfT circuitry overhead, and low correlation between consecutive test vectors. The survey confirmed that reducing switching activity at the Circuit Under Test inputs is the most effective approach to minimizing test mode power consumption.

Nourani proposed a low-transition test pattern generation technique specifically for BIST-based applications, published in IEEE Transactions on Computers. His approach targeted the reduction of average and peak power during testing by controlling the number of transitions between consecutive test vectors. Nourani demonstrated that low-transition patterns can significantly reduce the dynamic power consumption of the Circuit Under Test without requiring fundamental changes to the BIST architecture.

3.2.2 Modified LFSR Approaches

Girard et al. proposed a modified clock scheme where only half the LFSR flip-flops are active at any time, reducing switching transitions by 50% with minimal impact on fault coverage.

Katti, Ruan, and Khattri presented a multiple-output low-power Linear Feedback Shift Register design published in IEEE Transactions on Circuits and Systems. Their design introduced structural modifications to the LFSR to reduce the power consumed by the register itself during pattern generation, complementing approaches that focus on reducing the switching activity at the CUT inputs.

Wang and Gupta proposed the DS-LFSR, a BIST TPG specifically designed for low switching activity, published in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems. Their approach introduced a distance-sequence based reordering of LFSR outputs to minimize the Hamming distance between consecutive test vectors, thereby reducing the number of simultaneous bit transitions applied to the Circuit Under Test.

3.2.3 Gray Code Based Techniques

BOYE and Tian-Wang Li proposed a novel BIST scheme for low power testing that identified single-input-change sequences as the most effective mechanism for minimizing switching activity in test patterns. Their work demonstrated that when only one-bit changes between consecutive test vectors, the switching activity at the CUT inputs is minimized, directly reducing dynamic power consumption during testing.

He, Li, and Gong presented a scheme for low power BIST test pattern generation that combined pseudo-random sequences with structured low-transition patterns. Their approach demonstrated that hybrid test pattern generation strategies can achieve better power reduction than either purely pseudo-random or purely deterministic approaches alone.

Lei, Hou, Shao, and Liang established the theoretical basis for SIC circuits in BIST design, proving that XOR-combining an LFSR-generated seed with a Gray code sequence produces a single-input-change output sequence. This result directly motivates the LP-TPG architecture in the present work, where the LP-LFSR seed is XOR-combined with the Gray code generator output to produce low-transition test patterns.

Lei, Guo, Cao, Liu, and Wang proposed SACSR, a low power BIST method for sequential circuits, extending low power testing techniques from combinational circuits to sequential designs. Their work demonstrated that the principles of switching activity reduction can be applied to sequential circuit testing through appropriate modifications to the test pattern generation and application strategy.

3.3 Comparative Summary of Research Work

The following table presents a comparative summary of the key research contributions reviewed in this chapter, highlighting the techniques used and key findings reported.

Table 2: Comparative Summary of Research Work on Low Power Testing

Reference	Author(s)	Year	Technique	Key Finding
[1]	Balwinder Singh et al.	2009	LFSR power optimization	LFSR power optimization essential for low power BIST
[2]	Y. Zorian	1993	Distributed BIST control	Distributed BIST enables efficient multi-block testing
[3]	P. Girard	2002	Survey of LP testing	Four causes of high test power identified
[4]	M. Nourani	2008	Low-transition TPG	Low-transition patterns reduce average and peak power
[5]	BOYE and T.W. Li	2010	SIC based BIST	Single-input-change sequences minimize switching activity
[6]	R.S. Katti et al.	2006	Multiple-output LP-LFSR	Structural LFSR modifications reduce register power
[7]	P. Girard et al.	2001	Modified clock scheme	Half flip-flop activation reduces transitions by 50%
[8]	S. Wang and S.K. Gupta	2002	DS-LFSR	Distance-sequence reordering reduces Hamming distance
[9]	S.C. Lei et al.	2008	SACSR sequential BIST	SIC principles extended to sequential circuits
[10]	R.H. He et al.	2003	Hybrid LP-TPG scheme	Hybrid pseudo-random and structured patterns reduce power
[11]	S.C. Lei et al.	2008	SIC circuit theory	XOR of LFSR seed with Gray code produces SIC output
[12]	A. Kavitha et al.	2021	LP-LFSR with Gray code XOR	LP-TPG reduces switching transitions from 126,496 to 121,077 for 8×8 multiplier

3.4 Research Gap and Motivation

The literature review reveals that significant research has been conducted in the area of low power test pattern generation, with various techniques proposed to reduce switching activity and dynamic power consumption during BIST-based testing.

Conventional LFSR-based test pattern generators as described by Girard and Zorian remain the dominant choice in BIST implementations due to their simplicity and compact hardware, but they consistently produce high switching activity between consecutive test vectors due to the pseudo-random nature of their output sequences.

Modified clock schemes such as the approach proposed by Girard et al. reduce switching activity by deactivating half the LFSR flip-flops, but introduce gated clock architectures that create timing violations in modern FPGA implementation flows. DS-LFSR proposed by Wang and Gupta reduces switching by reordering the output sequence but requires complex reordering logic that increases hardware overhead.

Lei et al. prove that XOR-combining an LFSR seed with a Gray code sequence produces single-input-change outputs, forming the mathematical basis for the hybrid LP-TPG approach. However, prior implementations use NOR-gate clock gating to control LFSR advancement, creating gated clock paths that trigger critical timing warnings in modern EDA tools like Xilinx Vivado.

The proposed LP-TPG replaces NOR-gate clock gating with a synchronous clock enable within the always block, eliminating gated clock nets and achieving clean timing closure in Xilinx Vivado 2025.1. The design is implemented and verified on the Artix-7 FPGA (Basys3 board). The present work is inspired by and extends the LP-TPG architecture proposed by Kavitha et al, re-implementing it on the Artix-7 FPGA using Xilinx Vivado 2025.1 with synchronous clock enable logic in place of NOR-gate gating, achieving clean timing closure in modern FPGA design flows.

CHAPTER 4

METHODOLOGY

4.1 Overview of Methodology

The methodology of the proposed Low Power Test Pattern Generator (LP-TPG) focuses on developing an efficient test pattern generation mechanism for Built-In Self-Test (BIST) in digital circuits. Conventional testing techniques using standard Linear Feedback Shift Registers generate pseudo-random patterns that often lead to high switching activity in the Circuit Under Test, resulting in higher dynamic power consumption during testing.

To address this issue, the proposed system integrates a Low Power Linear Feedback Shift Register (LP-LFSR) with Gray code based single-input-change sequences. The LP-LFSR generates pseudo-random seed values while the Gray code generator produces deterministic patterns that ensure only one input changes between successive vectors. This significantly reduces transition activity and power dissipation during testing.

The generated patterns are combined through an XOR array and applied to the Circuit Under Test, which in this work is an 8×8 Braun Array Multiplier. The entire design is described using Verilog Hardware Description Language and implemented using Xilinx Vivado 2025.1 targeting the Artix-7 FPGA device (xc7a35tcbg236-1). Functional verification is performed through behavioral simulation, and power analysis is carried out to evaluate the efficiency of the proposed architecture compared to a conventional LFSR-based test pattern generator.

4.2 Methodology Flow

The complete methodology followed in this project consists of several sequential steps starting from conceptual study through to implementation, simulation, and power analysis.

Step 1 — Study of VLSI Testing and BIST:

The concepts of VLSI testing, fault models, and Built-In Self-Test architectures are studied to understand the challenges involved in testing modern digital circuits. The role of the Test Pattern Generator within the BIST framework is analyzed in detail.

Step 2 — Analysis of Conventional LFSR Testing:

The limitations of conventional LFSR-based test pattern generators are analyzed, with particular focus on the issue of high switching activity between consecutive test vectors and its impact on dynamic power consumption during test mode. A conventional LFSR-based TPG is designed and implemented as the baseline for comparison.

Step 3 — Design of LP-LFSR:

The Low Power Linear Feedback Shift Register is designed with a synchronous clock enable mechanism that allows the LFSR to advance only when the counter bit $c1[3]$ is logic zero, effectively causing the LP-LFSR to update once every 8 clock cycles instead of every cycle. This controlled advancement reduces unnecessary switching transitions in the LFSR output. The feedback polynomial $f[7] = f[1] \text{ XOR } f[0]$ is selected and the LFSR is initialized with a non-zero seed value.

Step 4 — Gray Code Generator Implementation:

An 8-bit Gray code generator is implemented using combinational XOR logic. The generator receives the output of an 8-bit binary counter and converts it into a Gray code sequence in which only one bit changes between consecutive values. This single-input-change property is the key mechanism for reducing switching activity in the final test patterns.

Step 5 — Integration of TPG Architecture:

The LP-LFSR output and the Gray code generator output are combined using an XOR array to produce the final low-transition test patterns. Two independent TPG instances (tpg01 and tpg02) are instantiated with different seed values to generate independent 8-bit test pattern streams for the two inputs of the 8×8 Braun Array Multiplier.

Step 6 — Verilog HDL Implementation:

The complete LP-TPG architecture including the LP-LFSR, counter, Gray code generator, XOR combination logic, and the Braun Array Multiplier as the Circuit Under Test is implemented using Verilog Hardware Description Language. Each functional block is designed as an independent module and integrated into a top-level module (tpgmain8).

Step 7 — Simulation using Xilinx Vivado:

Behavioral simulation is performed in Xilinx Vivado 2025.1 using a dedicated Verilog testbench (tb_tpgmain8). The testbench applies a 100 MHz clock and observes the 16-bit product output over a simulation duration of 2560 ns covering 256 test patterns. Simulation waveforms are analyzed to verify the functional correctness of the design.

Step 8 — Power Analysis and Comparison:

Post-implementation power analysis is performed using the Xilinx Vivado Power Analyzer with switching activity data captured from simulation using a SAIF (Switching Activity Interchange Format) file. The power consumption of the proposed LP-LFSR based TPG is compared against the conventional LFSR based TPG to demonstrate the effectiveness of the proposed low power architecture.

4.3 Methodology Flow Diagram

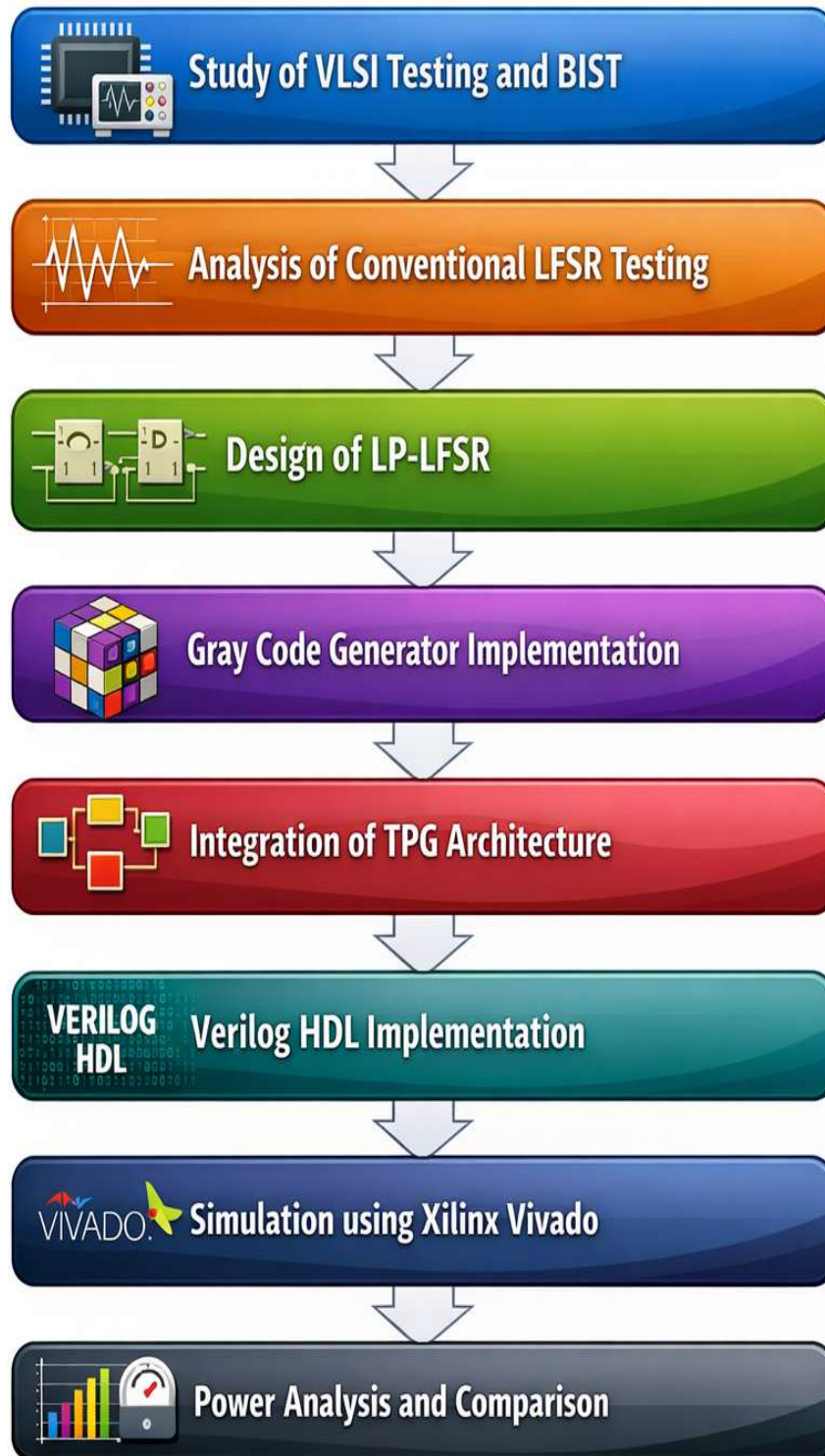


Figure 4:Methodology flow chart

4.4 Design Strategy

The design strategy of the proposed LP-TPG system is built around three core principles: power reduction through controlled switching, functional correctness through modular design, and practical implementability through FPGA-compatible architecture.

Power Reduction Through Controlled Switching:

The primary design decision is the replacement of the conventional full-speed LFSR with a clock-enabled LP-LFSR that advances only once every 8 clock cycles. During the intervening 7 cycles, the Gray code generator provides the changing component of the test pattern, ensuring that only one bit changes per clock cycle. This combination of slow-changing pseudo-random seeds with single-input-change Gray code sequences produces test patterns with significantly fewer simultaneous transitions compared to conventional LFSR outputs.

Functional Correctness Through Modular Design:

The architecture is divided into clearly defined independent modules — LP-LFSR, counter, Gray code generator, XOR array, and Braun Array Multiplier. Each module is independently verifiable through simulation, and the modular structure simplifies debugging and validation of the complete system. Two independent TPG instances with different seed values (0xAD for tpg01 and 0xAA for tpg02) are used to ensure decorrelated test pattern streams for the two inputs of the multiplier, improving fault coverage.

FPGA-Compatible Architecture:

A key design decision in this work is the replacement of the gated clock architecture used in previous LP-LFSR implementations with a synchronous clock enable mechanism. In previous designs, the LP-LFSR clock was gated using a combinational AND gate, which creates timing violations and critical warnings in modern EDA tools. In the proposed design, the clock enable condition ($\sim c1[3]$) is implemented as a conditional statement inside the synchronous always block, eliminating all gated clock nets and ensuring clean timing closure in Xilinx Vivado 2025.1. This makes the design fully compatible with FPGA implementation flows and eliminates timing-related synthesis warnings.

CHAPTER 5

FUNDAMENTAL CONCEPTS

5.1 Test Pattern Generator (TPG)

A Test Pattern Generator (TPG) is an essential component used in Built-In Self-Test (BIST) systems for testing digital circuits. The primary function of a TPG is to generate a sequence of input test vectors that are applied to the Circuit Under Test in order to stimulate its internal nodes and detect the presence of faults. These test patterns must be capable of activating different logic paths within the circuit to ensure effective fault detection coverage.

In conventional testing methods, test patterns are generated using external testing equipment and applied to the circuit through dedicated test pins. However, as the complexity of VLSI circuits increases, external testing becomes expensive, slow, and unable to test internal nodes that are not accessible from the primary inputs and outputs. To overcome this limitation, BIST techniques integrate the test pattern generator directly into the circuit so that test patterns can be generated and applied internally without external intervention.

5.1.1 Definition and Purpose

A Test Pattern Generator is a hardware module that produces input test vectors applied to a digital circuit for testing purposes. The main objective of a TPG is to stimulate different parts of the circuit so that faults present in the system can be detected through observation of the output responses. In BIST systems, the TPG is integrated directly within the circuit and generates test patterns autonomously during a dedicated test mode.

The generated test patterns must satisfy two key requirements. First, they must achieve high fault coverage by activating all logic paths and nodes within the circuit under test. Second, in low power testing applications, the patterns must minimize switching transitions between consecutive vectors to reduce dynamic power consumption during testing. The design of the TPG therefore involves a careful balance between fault coverage effectiveness and power efficiency.

5.1.2 Role of TPG in BIST

In a BIST architecture, the Test Pattern Generator plays the central role of driving the entire testing process. The TPG produces a continuous sequence of test vectors that are applied to the inputs of the Circuit Under Test. As the CUT processes these inputs, the corresponding output responses are captured by the Output Response Analyzer, which compares them against expected values or compresses them into a signature for fault detection.

The efficiency of a BIST system is directly determined by the quality of the test patterns generated by the TPG. Patterns that activate more logic paths and cover more fault scenarios result in higher fault coverage and more reliable testing. In low power BIST implementations, the TPG must additionally ensure that the generated patterns produce minimal switching activity at the inputs of the CUT, thereby reducing the dynamic power dissipation during the test mode.

5.2 Linear Feedback Shift Register (LFSR)

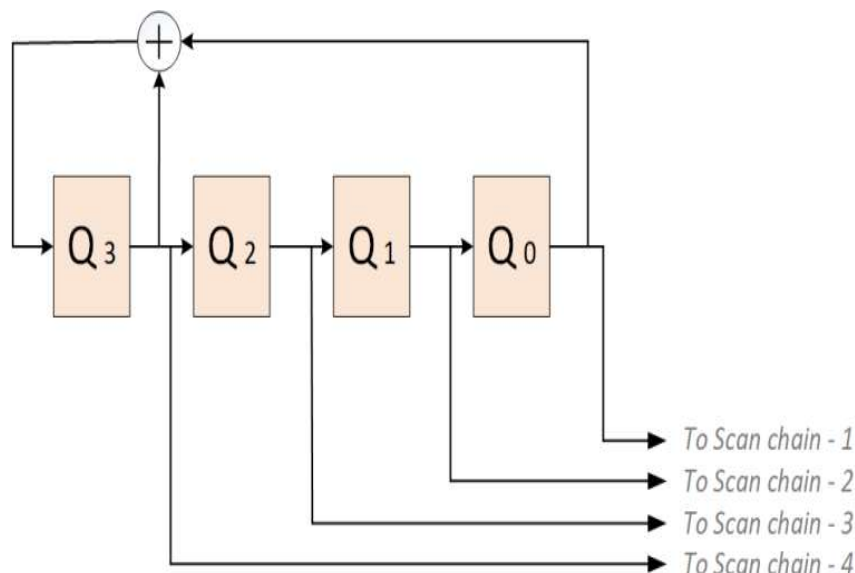


Figure 5: Example of LFSR

A Linear Feedback Shift Register is a sequential digital circuit widely used for generating pseudo-random sequences in digital systems. It consists of a series of flip-flops connected in a shift register configuration along with feedback logic implemented using XOR gates. The feedback signal is generated by combining the outputs of selected register stages, known as feedback taps, and feeding the result back to the input of the first register stage.

LFSRs are the most commonly used circuits for pseudo-random test pattern generation in BIST systems because they produce long sequences of binary patterns using simple and compact hardware. These pseudo-random sequences are effective for testing digital circuits because they apply a diverse range of input combinations to the Circuit Under Test, achieving good fault coverage with minimal hardware overhead.

5.2.1 Structure of LFSR

A Linear Feedback Shift Register consists of n flip-flops connected in a chain, where n is the bit width of the register. Each flip-flop stores one bit of the current register state. At every rising edge of the clock signal, the contents of each flip-flop are shifted to the adjacent stage, and a new

feedback bit computed through XOR operations on selected tap positions is inserted into the first flip-flop.

The structure of an LFSR is defined by its feedback polynomial, which specifies which register stages contribute to the XOR feedback computation. The selection of feedback taps determines the length and statistical properties of the generated sequence. A properly designed LFSR with an appropriate feedback polynomial can generate a maximum-length sequence, also known as an m-sequence, which cycles through all $2^n - 1$ possible non-zero states before repeating.

5.2.2 Working Principle

The operation of an LFSR begins with loading a non-zero initial value called the seed into the register. The seed serves as the starting state for the sequence generation. At each clock cycle, the register contents shift by one position and a new feedback bit is computed and inserted at the first stage.

For the conventional LFSR used as the baseline in this work, the feedback polynomial is $x^8 + x^6 + x^5 + x^4 + 1$, implemented using taps at positions [8, 6, 5, 4]. This produces a maximal-length sequence of 255 patterns before repeating. The register is initialized with the seed value 8'b10101010 and advances at every clock cycle, producing a new pseudo-random output pattern at each clock edge.

5.2.3 Feedback Polynomial

The feedback polynomial defines the mathematical relationship between the tap positions used in the XOR feedback computation of the LFSR. For an n-bit LFSR, a primitive polynomial of degree n must be selected to ensure maximum-length sequence generation.

The conventional LFSR used in this work implements the polynomial $x^8 + x^6 + x^5 + x^4 + 1$, which is a known primitive polynomial for an 8-bit maximal-length LFSR. The Verilog implementation of this feedback is as follows:

```
f[7] <= f[0];
```

```
f[5] <= f[6] ^ f[0];
```

```
f[4] <= f[5] ^ f[0];
```

```
f[3] <= f[4] ^ f[0];
```

The LP-LFSR used in the proposed design implements a simplified feedback polynomial $f[7] = f[1] \text{ XOR } f[0]$, with the seed value 8'b10101101 for tpg01 and 8'b10101010 for tpg02. The LP-LFSR advances only when the clock enable condition ($\sim c1[3]$) is satisfied, meaning it updates once every 8 clock cycles rather than every cycle. This controlled advancement is the key mechanism that reduces switching transitions in the proposed architecture.

5.2.4 Limitations of Conventional LFSR

Although LFSRs are efficient and widely used for pseudo-random test pattern generation, conventional LFSR-based TPGs have several important limitations when applied to low power testing:

- **High switching activity:** Since the generated sequences are pseudo-random, multiple bits may change simultaneously between successive output vectors. In an 8-bit conventional LFSR, the average number of bit transitions per clock cycle is approximately 4, which represents 50% of all bits toggling every cycle.
- **No transition control:** Conventional LFSRs advance at every clock cycle and have no mechanism to control the correlation between successive output vectors. This means the number of transitions between consecutive patterns cannot be minimized without modifying the architecture.
- **Increased dynamic power:** The high switching activity produced by conventional LFSR patterns directly increases the dynamic power consumption of the Circuit Under Test during testing, potentially exceeding safe operating power limits.
- **Reliability risks:** Excessive switching activity during testing can cause overheating, IR drop, and electromigration effects that degrade the reliability of the circuit under test and reduce its operational lifespan.

5.3 Gray Code and Single-Input-Change Sequences

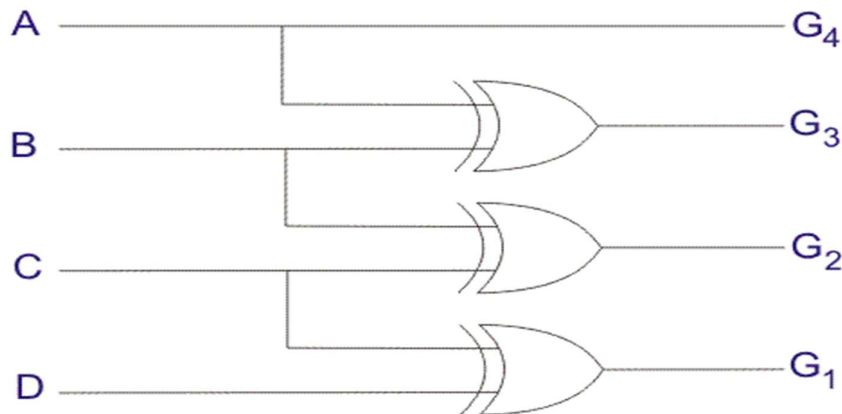


Figure 6: Basic example of gray code convertor

Gray code is a binary numeral system in which two consecutive values differ in exactly one bit position. This property is known as the Single-Input-Change (SIC) characteristic. Unlike

conventional binary counting, where multiple bits may change simultaneously between successive values, Gray code guarantees that only one bit changes at a time regardless of the counter value.

In digital circuit testing, switching activity plays a major role in determining dynamic power consumption. When several bits change simultaneously between consecutive input patterns, a large number of internal nodes in the circuit switch at the same time, resulting in high dynamic power dissipation. Single-input-change sequences such as Gray code sequences are therefore highly effective for reducing switching activity in low power test pattern generation.

5.3.1 Gray Code Conversion

Gray code can be generated from a binary number using a simple XOR-based conversion method. For an n -bit binary number $B = (B_{n-1}, B_{n-2}, \dots, B_1, B_0)$, the corresponding Gray code $G = (G_{n-1}, G_{n-2}, \dots, G_1, G_0)$ is computed as follows:

$$G_{n-1} = B_{n-1}$$

$$G_i = B_{i+1} \text{ XOR } B_i \text{ for } i = n-2 \text{ down to } 0$$

The 8-bit Gray code generator implemented in this work uses the following combinational logic, as defined in gray1.v:

```
g[7] <= count[7];
g[6] <= count[7] ^ count[6];
g[5] <= count[6] ^ count[5];
g[4] <= count[5] ^ count[4];
g[3] <= count[4] ^ count[3];
g[2] <= count[3] ^ count[2];
g[1] <= count[2] ^ count[1];
g[0] <= count[1] ^ count[0];
```

This implementation uses seven XOR operations to convert the 8-bit binary counter output into an 8-bit Gray code sequence, ensuring that only one bit changes between consecutive output values.

5.3.2 Why SIC Sequences Reduce Power

In CMOS digital circuits, dynamic power consumption is proportional to the switching activity factor α , which represents the average number of signal transitions per clock cycle. Single-

input-change (SIC) sequences minimize α by ensuring that only one bit of the input pattern changes between consecutive clock cycles.

When a Gray code sequence is applied to a circuit under test, at most one primary input changes per clock cycle. This means that only the logic paths connected to that single changing input are activated during each clock transition, while all other paths remain stable. As a result, the overall switching activity within the circuit is significantly reduced compared to applying pseudo-random patterns where multiple inputs may change simultaneously.

By combining Gray code sequences with LP-LFSR pseudo-random seeds through XOR combination, the proposed LP-TPG architecture achieves both effective fault coverage from the pseudo-random component and reduced switching activity from the single-input-change component.

Furthermore, the reduction in switching activity directly leads to lower dynamic power dissipation, reduced heat generation, and improved reliability of the circuit during testing. This is particularly important in large-scale VLSI systems, where excessive switching can cause power integrity issues such as IR drop and ground bounce.

5.4 Braun Array Multiplier as Circuit Under Test

In this work, an 8×8 Braun Array Multiplier is used as the Circuit Under Test to evaluate the effectiveness of the proposed Low Power Test Pattern Generator. The Braun Array Multiplier is a well-known parallel multiplier architecture for unsigned binary multiplication that is widely used as a benchmark circuit in VLSI testing research due to its regular structure, significant logic complexity, and substantial switching activity during operation.

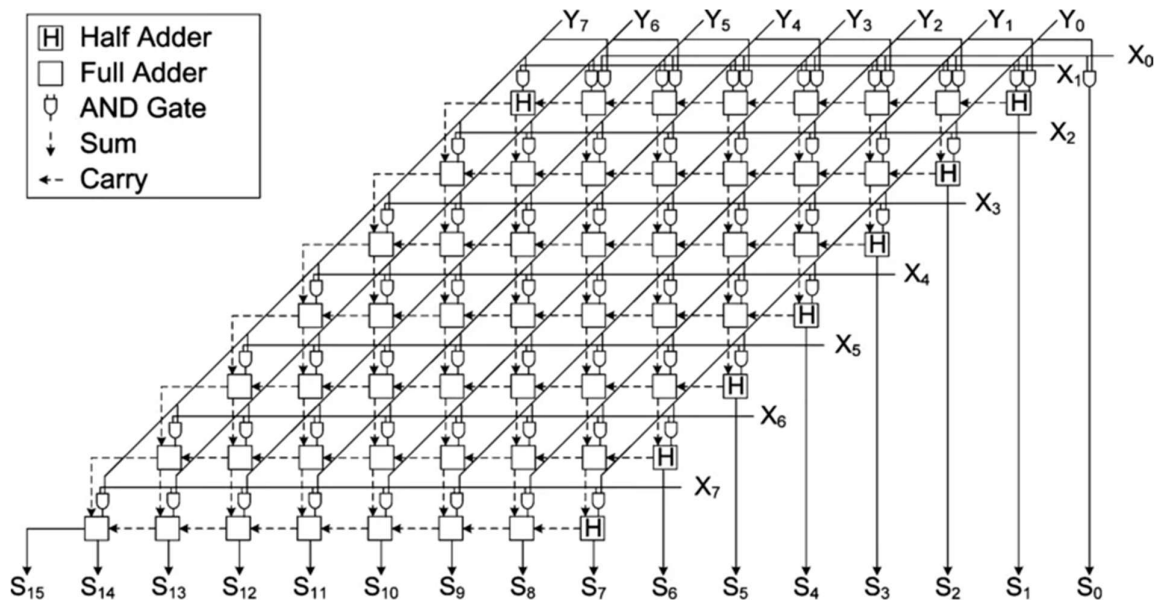


Figure 7: Basic Braun array multiplier

5.4.1 Structure of Braun Array Multiplier

The Braun Array Multiplier performs multiplication of two n -bit unsigned binary numbers by first generating all partial products using AND gates and then summing them using an array of half adders and full adders arranged in a regular grid pattern.

For an 8×8 multiplier, the structure consists of the following components:

- **Partial product generation:** 64 AND gates generate all partial products by ANDing each bit of the multiplicand $a[7:0]$ with each bit of the multiplier $b[7:0]$, producing 64 partial product bits $x[63:1]$ along with $p[0] = a[0] \& b[0]$.
- **Adder array:** The partial products are summed using a structured array of half adders (ha1) and full adders (fa1) arranged in 14 rows. Half adders handle the boundary positions where only two inputs are available, while full adders handle the interior positions where three inputs must be summed.
- **Output:** The 8×8 multiplication produces a 16-bit product $p[15:0]$, representing all possible output values from $0 \times 0 = 0$ to $255 \times 255 = 65025$.

The implemented Braun Array Multiplier in this work uses 63 partial product wires, 55 carry wires, and 42 intermediate sum wires, as defined in arraymultiplier01.v.

5.4.2 Role as CUT

The 8×8 Braun Array Multiplier serves as the Circuit Under Test in this project for several important reasons. First, its regular and well-understood structure makes it straightforward to verify correct operation through simulation. Second, its significant logic depth and large number of interconnected gates produce substantial switching activity when test patterns are applied, making it an effective benchmark for evaluating the power reduction achieved by the proposed LP-TPG architecture. Third, multipliers are fundamental building blocks of digital signal processing and arithmetic systems, making them a practically relevant choice as a test circuit.

The test patterns generated by the proposed LP-TPG are applied to the $a[7:0]$ and $b[7:0]$ inputs of the multiplier through two independent TPG instances (tpg01 and tpg02), and the resulting 16-bit product output is observed during simulation to verify correct operation of the complete system.

CHAPTER 6

PROPOSED LP-TPG ARCHITECTURE

6.1 Overview of Proposed System

The proposed Low Power Test Pattern Generator (LP-TPG) architecture is designed to address the high switching activity problem associated with conventional LFSR-based test pattern generators. The system generates test patterns with significantly reduced bit transitions between consecutive vectors by combining pseudo-random sequences produced by a Low Power Linear Feedback Shift Register with single-input-change sequences produced by a Gray code generator. The final test patterns are obtained by XOR combining the LP-LFSR output with the Gray code output, resulting in a pattern stream that retains the randomness required for effective fault coverage while significantly reducing the switching activity applied to the Circuit Under Test.

The complete LP-TPG system consists of two identical TPG instances, tpg01 and tpg02, each generating an independent 8-bit test pattern stream. The outputs of tpg01 and tpg02 are applied to the a[7:0] and b[7:0] inputs respectively of an 8×8 Braun Array Multiplier, which serves as the Circuit Under Test. The two TPG instances are initialized with different seed values to ensure that the test patterns applied to the two multiplier inputs are decorrelated, improving the overall fault coverage of the testing process.

The entire system is implemented in Verilog HDL, synthesized and implemented in Xilinx Vivado 2025.1, and targeted at the Artix-7 FPGA device (xc7a35tcpg236-1). Functional verification is performed through behavioral simulation and power consumption is evaluated using post-implementation power analysis with SAIF-based switching activity annotation.

6.2 LP-LFSR Design and Algorithm

The Low Power Linear Feedback Shift Register is the core innovation of the proposed architecture. Unlike a conventional LFSR that advances at every clock cycle and produces a new pseudo-random output at every clock edge, the LP-LFSR incorporates a synchronous clock enable mechanism that controls the rate at which the register advances. This controlled advancement reduces the number of transitions in the LFSR output, thereby reducing the contribution of the pseudo-random component to overall switching activity.

6.2.1 LP-LFSR Feedback Polynomial

The LP-LFSR in this work implements a simplified feedback polynomial defined as:

$$f[7] = f[1] \text{ XOR } f[0]$$

At each enabled clock cycle, the register shifts its contents by one position and the new feedback bit $f[7]$ is computed as the XOR of the two least significant bits $f[1]$ and $f[0]$. The remaining bits shift as follows:

$$f[7] \leftarrow f[1] \oplus f[0];$$

$$f[6] \leftarrow f[7];$$

$$f[5] \leftarrow f[6];$$

$$f[4] \leftarrow f[5];$$

$$f[3] \leftarrow f[4];$$

$$f[2] \leftarrow f[3];$$

$$f[1] \leftarrow f[2];$$

$$f[0] \leftarrow f[1];$$

This feedback structure produces a pseudo-random sequence that serves as the seed component of the final test pattern. The simplified polynomial with only two tap positions reduces the combinational logic required for feedback computation compared to a full maximal-length polynomial, contributing to the overall low power objective of the design.

6.2.2 Seed Initialization

The LP-LFSR is initialized with a non-zero seed value upon assertion of the asynchronous reset signal. Two different seed values are used for the two TPG instances to ensure decorrelated pattern generation:

Table 3: TPGs seed

	Seed Value (Binary)	Seed Value (Hex)
tpg01 (LP-LFSR for input a)	10101101	0xAD
tpg02 (LP-LFSR for input b)	10101010	0xAA

The use of different seeds for the two instances ensures that the test patterns applied to the a and b inputs of the Braun Array Multiplier are independent of each other. If both instances used the same seed, the multiplier inputs would be identical at every clock cycle, significantly limiting the range of input combinations tested and reducing fault coverage.

6.2.3 Clock Enable Mechanism

The key architectural feature that distinguishes the LP-LFSR from a conventional LFSR is the synchronous clock enable mechanism. In the proposed design, the LP-LFSR advances only when bit 3 of the counter output $c1[3]$ is logic zero. Since the counter increments by one at every clock cycle, $c1[3]$ is logic zero for 8 consecutive cycles and logic one for the next 8 cycles in a repeating pattern. This means the LP-LFSR updates its state once every 8 clock cycles, while during the remaining 7 cycles it holds its current value.

The clock enable condition is implemented as follows in Verilog:

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        c3 <= 8'b10101101;
    end else if (~c1[3]) begin
        c3[7] <= c3[1] ^ c3[0];
        c3[6] <= c3[7];
        // ... shift register logic
    end
end
```

This synchronous clock enable approach replaces the gated clock architecture used in previous LP-LFSR implementations. Gated clock designs create combinational clock paths that produce timing violations and critical warnings in modern EDA tools including Xilinx Vivado. The synchronous clock enable implementation eliminates all gated clock nets and achieves clean timing closure without any timing-related synthesis warnings.

6.2.4 LP-LFSR Algorithm Steps

The complete operation of the LP-LFSR based TPG follows these steps:

Step 1: Assert reset — LP-LFSR loads seed value, counter resets to zero, Gray code output resets to zero.

Step 2: Release reset — counter begins incrementing at every clock cycle.

Step 3: Gray code generator converts counter output to Gray code sequence $g[7:0]$ — one bit changes per cycle.

Step 4: Check clock enable condition — if $c1[3] = 0$, LP-LFSR advances to next state; if $c1[3] = 1$, LP-LFSR holds current state.

Step 5: XOR combine LP-LFSR output $c3[7:0]$ with Gray code output $g[7:0]$ to produce final test pattern $y[7:0]$.

Step 6: Apply $y[7:0]$ to Circuit Under Test input.

Step 7: Repeat Steps 2 through 6 for all test patterns.

6.3 Architecture Components

6.3.1 LP-LFSR Module

The LP-LFSR module implements the low power linear feedback shift register with synchronous clock enable. It is an 8-bit register with asynchronous reset and a conditional clock enable based on counter bit $c1[3]$. The module is instantiated twice within each TPG instance — once as part of `tpg01` with seed `0xAD` and once as part of `tpg02` with seed `0xAA`. In the synthesized design, the LP-LFSR registers are implemented as alternating FDCE (flip-flop with clock enable and asynchronous clear) and FDPE (flip-flop with clock enable and asynchronous preset) primitives on the Artix-7 FPGA, confirming that Vivado correctly inferred the clock enable structure from the Verilog description.

6.3.2 M-Bit Counter

The counter module implements an 8-bit synchronous binary counter with synchronous active-high reset. The counter increments by one at every rising clock edge and cycles from 0 to 255 before wrapping back to zero. The counter output $c1[7:0]$ serves two purposes: it provides the binary input to the Gray code generator, and its bit $c1[3]$ provides the clock enable signal for the LP-LFSR. Two independent counter instances (`count1` for `tpg01` and `count01` for `tpg02`) are used, both initialized to zero upon reset.

```
always @(posedge clk) begin
```

```
    if (rst)
```

```
        count <= 8'b0;
```

```
    else
```

```
        count <= count + 1'b1;
```

```
end
```

6.3.3 Gray Code Generator

The Gray code generator module converts the 8-bit binary counter output into an 8-bit Gray code sequence using combinational XOR logic. The module is implemented as a purely combinational always @* block, ensuring zero latency conversion from binary to Gray code at every clock cycle. The most significant bit of the Gray code is equal to the most significant bit of the binary counter, while each remaining bit is computed as the XOR of two adjacent binary counter bits.

The critical property of the Gray code output is that only one bit changes between consecutive values as the counter increments. This single-input-change property is directly transferred to the final test pattern output through the XOR combination with the LP-LFSR, ensuring that the number of simultaneous transitions in the final test patterns is minimized.

6.3.4 XOR Array

The XOR array combines the LP-LFSR output $c3[7:0]$ and the Gray code output $g[7:0]$ bit-by-bit to produce the final 8-bit test pattern $y[7:0]$. This operation is implemented as a simple concurrent assignment:

```
assign y = c2 ^ c3;
```

The XOR combination serves two purposes. First, it introduces the pseudo-random component from the LP-LFSR into the final pattern, ensuring that the test sequence has sufficient randomness to achieve good fault coverage. Second, since the LP-LFSR changes slowly (once every 8 cycles) while the Gray code changes by one bit every cycle, the dominant contribution to pattern changes in the output comes from the Gray code component. This means the final test patterns inherit the single-input-change characteristic of the Gray code for most clock cycles, with occasional larger transitions occurring when the LP-LFSR updates every 8 cycles

6.4 Block Diagram

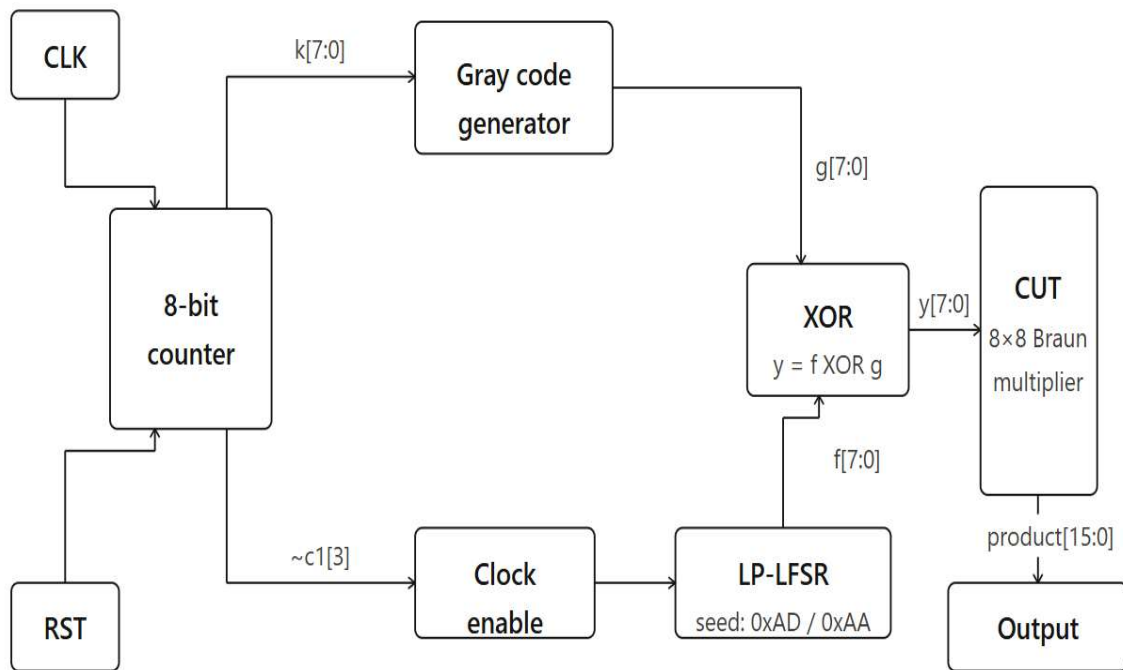


Figure 8:LOW POWER TPG BLOCK DIAGRAM

The block diagram of the proposed LP-TPG architecture illustrates the signal flow from the clock input through the LP-LFSR and the Gray code generation path to the final XOR combination that produces the test pattern output $x[m-1:0]$. The CLK signal drives both the LP-LFSR through a synchronous clock enable condition and the m -bit counter directly. The counter output $k[m-1:0]$ feeds the Gray code generator, which produces the single-input-change sequence $g[m-1:0]$. The LP-LFSR produces the pseudo-random sequence $f[m-1:0]$. Both outputs are combined through the XOR array to produce the final m -bit test pattern $x[m-1:0]$ that is applied to the Circuit Under Test.

6.5 System Flow Chart

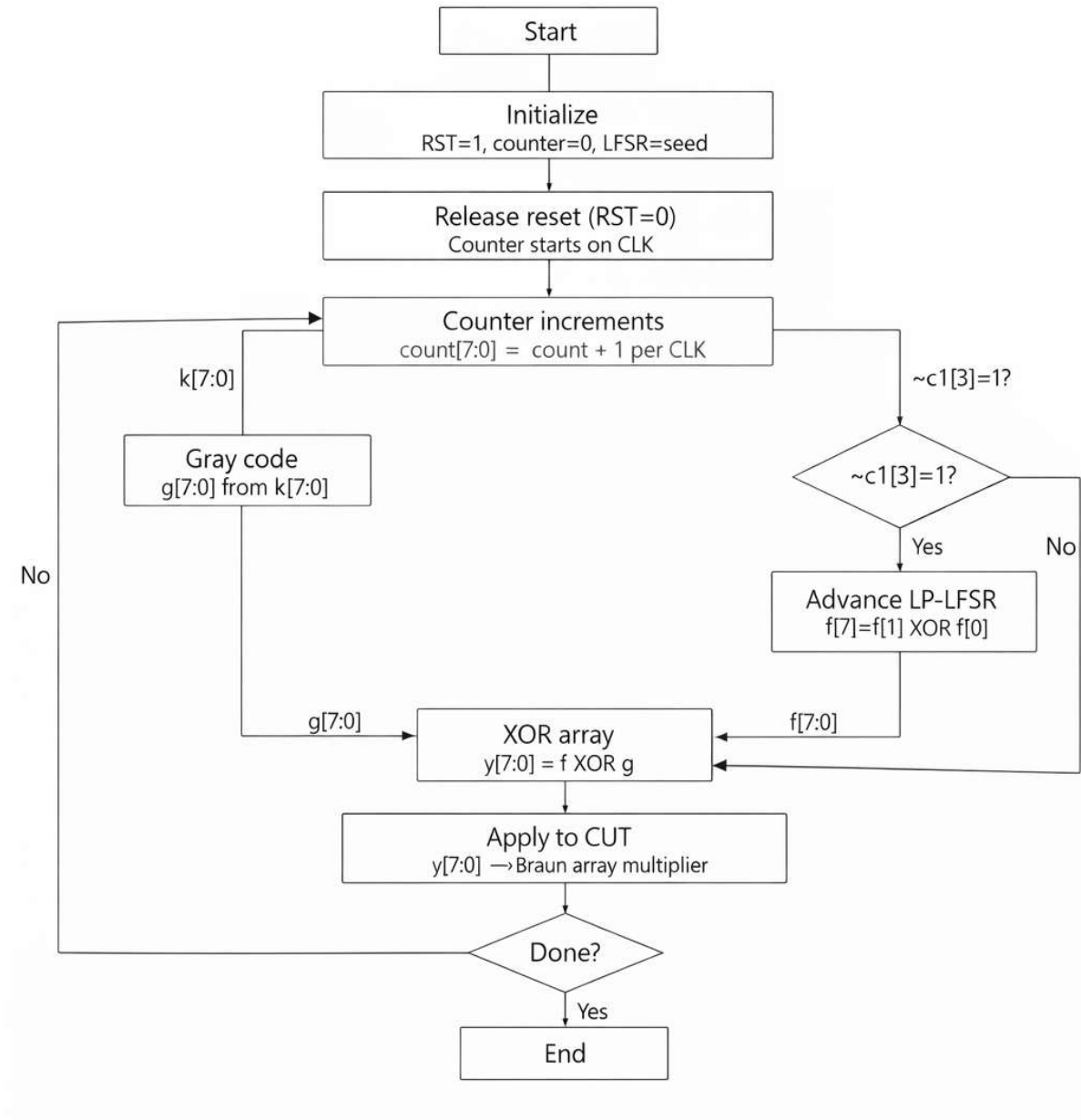


Figure 9:LOW POWER TPG ALGORITHM FLOWCHART

The system flowchart describes the step-by-step operation of the LP-TPG architecture. The process begins with the Start state, followed by initialization of the counter, LP-LFSR, and Gray code converter. The counter $k=[n-1:0]$ is then incremented at each clock cycle. The counter value simultaneously feeds two parallel paths: the clock gated LP-LFSR that initializes with its seed value at $k=0$, and the Gray code converter that produces single-input-change sequences. The outputs of both paths are combined through the XOR operation to produce the TPG output, which is then applied to the Circuit Under Test — the 8×8 Braun Array Multiplier.

CHAPTER 7

TOOLS AND TECHNOLOGIES

7.1 Target FPGA Platform — Artix-7 (xc7a35tcpg236-1)

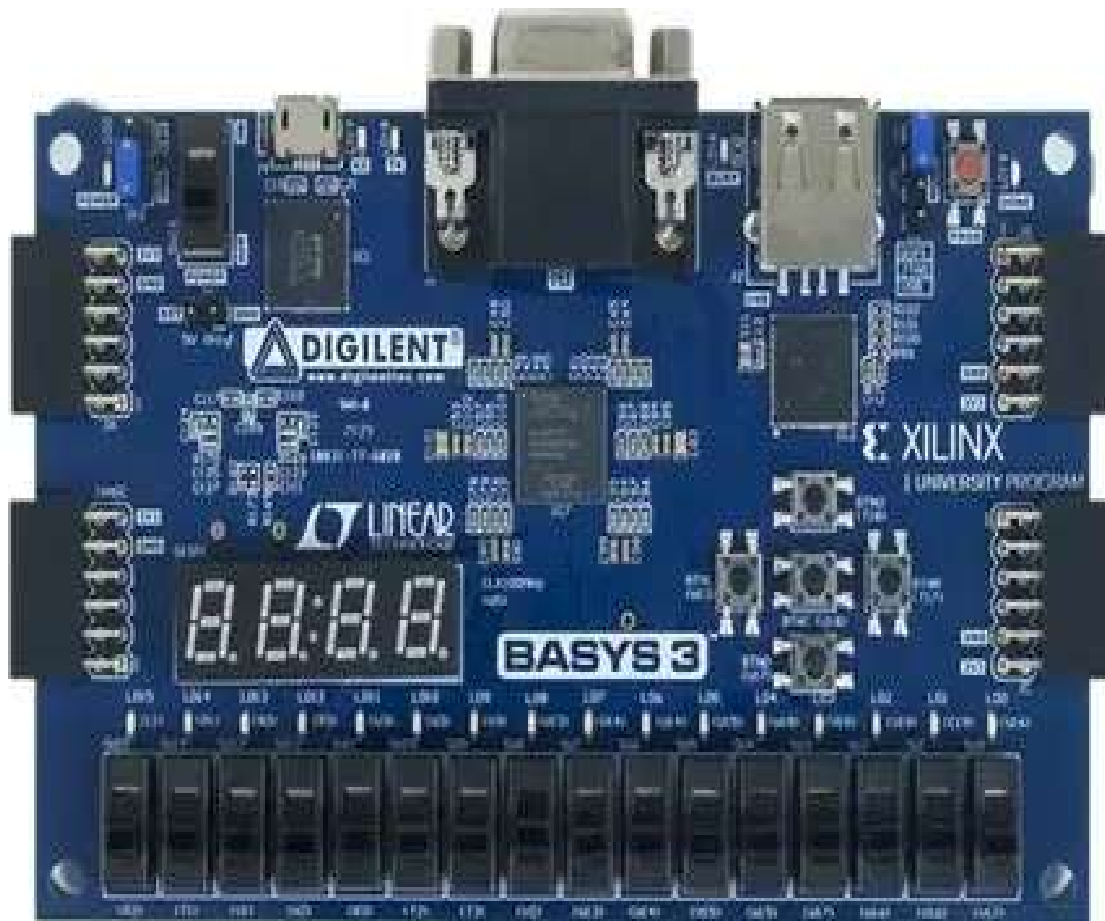


Figure 10: Xilinx Artix-7 FPGA Board

The proposed LP-TPG design is targeted at the Xilinx Artix-7 FPGA device with part number xc7a35tcpg236-1. The Artix-7 family is part of Xilinx's 7-series FPGA product line, manufactured using a 28nm high-performance low-power (HPL) process technology. The Artix-7 series is specifically optimized for cost-sensitive and power-sensitive applications, making it a highly appropriate target platform for evaluating a low power test pattern generator design.

The specific device used in this work is mounted on the Basys3 development board, which is one of the most widely used FPGA development platforms in academic and research environments. The Basys3 board provides a accessible and practical platform for FPGA-based digital design projects.

The key specifications of the target device are as follows:

Table 4: Artix-7 (xc7a35tcbg236-1) Device Specifications

Resource	Specification
Device Family	Artix-7
Part Number	xc7a35tcbg236-1
Process Technology	28nm HPL
Slice LUTs	20,800
Slice Registers	41,600
Slices	8,150
Block RAM	50 (36Kb each)
DSP Slices	90
Bonded IOBs	106
BUFGCTRL	32
Maximum Frequency	450 MHz
Package	CPG236 (236 pins)
Speed Grade	-1

The Artix-7 device provides significantly more resources than required by the proposed LP-TPG design, which uses less than 1% of available Slice LUTs and Slice Registers. This confirms that the proposed architecture is extremely lightweight and can be easily integrated as a BIST component within a much larger VLSI system on the same device.

7.2 Verilog HDL

Verilog Hardware Description Language is the hardware description language used for implementing all modules of the proposed LP-TPG system in this work. Verilog is a standardized IEEE 1364 hardware description language that allows digital circuits to be described at multiple levels of abstraction including behavioral, register transfer level (RTL), and structural levels.

Verilog supports both sequential and combinational logic in a unified syntax, making it ideal for designing circuits ranging from simple gates to complex state machines. It is synthesizable using modern EDA tools like Xilinx Vivado, enabling the same HDL description to be used for simulation, synthesis, and FPGA/ASIC implementation. In this work, all modules including the LP-LFSR, counter, Gray code generator, TPG integration modules, top-level module, Braun Array Multiplier,

and testbench are written in synthesizable Verilog HDL at the register transfer level. The design follows standard RTL coding guidelines to ensure correct synthesis and implementation in Xilinx Vivado 2025.1.

The key design modules and their Verilog file names are listed in the following table:

Table 5: Verilog Modules and File Names

Module Name	File Name	Function
lfsr1	lfsr1.v	LP-LFSR for TPG1 — seed 0xAD
lfsr01	lfsr01.v	LP-LFSR for TPG2 — seed 0xAA
count1	count1.v	8-bit counter for TPG1
count01	count01.v	8-bit counter for TPG2
gray1	gray1.v	Gray code generator for TPG1
gray01	gray01.v	Gray code generator for TPG2
tpg01	tpg01.v	TPG integration module 1
tpg02	tpg02.v	TPG integration module 2
tpgmain8	tpgmain8.v	Top-level LP-TPG module
arraymultiplier01	arraymultiplier01.v	8×8 Braun Array Multiplier (CUT)
ha1	ha1.v	Half adder primitive
fa1	fa1.v	Full adder primitive
tb_tpgmain8	tb_tpgmain8.v	Testbench for LP-TPG
lfsr_conv	lfsr_conv.v	Conventional LFSR module
tpg_conv1	tpg_conv1.v	Conventional TPG module 1
tpg_conv2	tpg_conv2.v	Conventional TPG module 2
tpgmain8_conv	tpgmain8_conv.v	Top-level conventional TPG module
tb_tpgmain8_conv	tb_tpgmain8_conv.v	Testbench for conventional TPG

7.3 Xilinx Vivado 2025.1

Xilinx Vivado Design Suite 2025.1 is the Electronic Design Automation (EDA) tool used for all stages of design implementation in this work including RTL analysis, synthesis, implementation, simulation, and power analysis. Vivado is the primary FPGA design tool for all Xilinx 7-series and UltraScale devices and provides an integrated environment for the complete digital design flow from HDL entry to bitstream generation.

Vivado 2025.1 offers several important capabilities that are utilized in this project. The Vivado Simulator supports behavioral, functional, and timing simulation of Verilog designs, enabling functional verification of the LP-TPG system through waveform analysis. The Vivado Synthesis engine converts the RTL Verilog description into a gate-level netlist optimized for the target Artix-7 device. The Vivado Implementation tool performs placement and routing of the synthesized netlist onto the physical FPGA resources. The Vivado Power Analyzer computes static and dynamic power estimates based on the implemented design and switching activity data provided through a SAIF file.

The installation of the Xilinx Vivado Design Suite is performed by following the steps given below:

1. Download the Vivado installer from the official AMD Xilinx website.
2. Extract the downloaded installation files to a suitable location on the system.
3. Run the Vivado setup executable file to start the installation process.
4. Select the required edition of the Vivado Design Suite.
5. Choose the device support packages required for the target FPGA device.
6. Select the installation directory where the software will be installed.
7. Complete the installation process by following the instructions provided by the setup wizard.
8. After successful installation, launch the Vivado Design Suite from the system application menu.

7.3.1 Project Creation

The steps involved in creating a new project in the Xilinx Vivado Design Suite are described below:

1. Launch the Vivado Design Suite from the system applications menu.
2. In the Vivado start window, select the ****Create New Project**** option.
3. Enter the project name and specify the location where the project files will be stored.
4. Select ****RTL Project**** as the project type.
5. Add the Verilog source files containing the design modules to the project.
6. Add the testbench file used for simulation and verification.
7. Select the target FPGA device for the project implementation.
8. Review the project summary and click ****Finish**** to create the project.

The Vivado project for the LP-TPG design is created with the following configuration:

Table 6:Project Configuration

Parameter	Value
Project Name	LP TPG USING LP LFSR
Target Device	xc7a35tcpg236-1
Design Language	Verilog
Top Module (LP-TPG)	tpgmain8
Top Module (Conv-TPG)	tpgmain8_conv
Simulation Top	tb_tpgmain8 / tb_tpgmain8_conv
Vivado Version	2025.1

7.3.2 Design Execution Flow

The complete design execution flow in Xilinx Vivado 2025.1 for this project consists of the following stages:

- **RTL Analysis:** The Verilog source files are analyzed for syntax correctness and the design hierarchy is established. The RTL schematic is generated to visualize the module interconnections and internal signal routing at the register transfer level.
- **Behavioral Simulation:** The testbench is executed using the Vivado Simulator to verify the functional correctness of the LP-TPG design. The simulation applies a 100 MHz clock and observes the 16-bit product output over 2560 ns. The SAIF file is generated during simulation to capture switching activity data for power analysis.
- **Synthesis:** The RTL design is synthesized into a technology-mapped gate-level netlist targeting the Artix-7 device. Vivado maps the Verilog RTL constructs to FPGA primitives including LUTs, flip-flops, carry chains, and I/O buffers. The synthesis schematic and utilization report are generated at this stage.
- **Implementation:** The synthesized netlist is placed and routed onto the physical Artix-7 FPGA resources. Vivado assigns specific LUT and flip-flop locations and routes all signal connections through the FPGA routing fabric. The implemented design is used as the basis for post-implementation power analysis.
- **Power Analysis:** The Vivado Power Analyzer is invoked on the implemented design with the SAIF switching activity file loaded to perform accurate post-implementation power estimation. The power report provides a breakdown of static and dynamic power consumption across all design components.

7.4 Testbench Design

The testbench modules `tb_tpgmain8` and `tb_tpgmain8_conv` are written in Verilog HDL to verify the LP-TPG and conventional TPG designs through behavioral simulation in Xilinx Vivado 2025.1. Both follow an identical structure to ensure a fair and directly comparable simulation environment. The key testbench parameters are as follows:

Table 7: Testbench Configuration Parameters

Parameter	LP-TPG Testbench	Conv-TPG Testbench
Module Name	<code>tb_tpgmain8</code>	<code>tb_tpgmain8_conv</code>
Clock Period	10 ns (100 MHz)	10 ns (100 MHz)
Reset Duration	100 ns	100 ns
Simulation Runtime	2560 ns	2560 ns
Test Patterns	256	256
Output Observed	<code>product[15:0]</code>	<code>product[15:0]</code>

The testbench generates a 100 MHz clock by toggling every 5 ns. The reset remains high for 100 ns to initialize registers, then goes low to start test pattern generation. The simulation runs for 2560 ns after reset, producing 256 test patterns for the Braun Array Multiplier. The \$monitor task logs time, reset, and 16-bit output continuously.

7.5 Complete Design Flow

The following table summarizes the complete design flow from Verilog source entry to final power analysis results:

Table 8: Design flow summary

Stage	Tool/Method	Output
HDL Entry	Verilog RTL coding	.v source files
RTL Analysis	Vivado RTL Analysis	RTL Schematic
Behavioral Simulation	Vivado Simulator	Waveform + SAIF file
Synthesis	Vivado Synthesis	Gate-level netlist + Utilization Report
Implementation	Vivado Implementation	Placed and routed design
Power Analysis	Vivado Power Analyzer + SAIF	Power Report
Comparison	LP-TPG vs Conv-TPG	Power reduction results

CHAPTER 8

RTL DESIGN AND IMPLEMENTATION

8.1 Module Hierarchy

The complete LP-TPG design is organized as a hierarchical Verilog design with a clearly defined top-level module and multiple sub-modules instantiated within it. The hierarchical structure separates each functional block into an independent module, making the design modular, readable, and easily verifiable through simulation.

The complete module hierarchy of the LP-TPG design is as follows:

Table 9:LP-TPG Module Hierarchy

Level	Module Name	File	Description
Top	tpgmain8	tpgmain8.v	Top-level integration module
Level 1	tpg01	tpg01.v	TPG instance 1 — generates a[7:0]
Level 1	tpg02	tpg02.v	TPG instance 2 — generates b[7:0]
Level 1	arraymultiplier01	arraymultiplier01.v	8×8 Braun Array Multiplier (CUT)
Level 2	count1	count1.v	8-bit counter inside tpg01
Level 2	gray1	gray1.v	Gray code generator inside tpg01
Level 2	count01	count01.v	8-bit counter inside tpg02
Level 2	gray01	gray01.v	Gray code generator inside tpg02
Level 2	ha1	ha1.v	Half adder inside arraymultiplier01
Level 2	fa1	fa1.v	Full adder inside arraymultiplier01

The LP-LFSR logic is directly embedded within tpg01 and tpg02 as a synchronous always block with clock enable, rather than as a separate instantiated module. This design decision ensures that Vivado correctly infers the clock enable structure during synthesis and maps the LP-LFSR registers to FDCE and FDPE flip-flop primitives without generating gated clock warnings.

8.2 Verilog Module Design

8.2.1 LP-LFSR Module (lfsr1 / lfsr01)

The LP-LFSR module implements an 8-bit linear feedback shift register with asynchronous reset and a simplified feedback polynomial $f[7] = f[1] \text{ XOR } f[0]$. The register shifts its contents by one position at each enabled clock edge and computes the new feedback bit from the two least significant bits. The key feedback logic is as follows:

```
f[7] <= f[1] ^ f[0]; // feedback polynomial
```

```
f[6] <= f[7];
```

```
f[5] <= f[6];
```

```
// ... shift continues down to f[0]
```

Two instances are used with different seed values — lfsr1 with seed 8'b10101101 (0xAD) for tpg01 and lfsr01 with seed 8'b10101010 (0xAA) for tpg02 — ensuring decorrelated test pattern streams for the two multiplier inputs.

8.2.2 Counter Module (count1 / count01)

The counter module implements an 8-bit synchronous binary counter with synchronous active-high reset. It increments by one at every rising clock edge and wraps from 255 back to 0. The core counting logic is:

```
if (rst)
```

```
    count <= 8'b0;
```

```
else
```

```
    count <= count + 1'b1;
```

The counter output serves two purposes: it provides the binary input to the Gray code generator, and its bit c1[3] provides the clock enable signal for the LP-LFSR.

8.2.3 Gray Code Generator Module (gray1 / gray01)

The Gray code generator converts the 8-bit binary counter output into an 8-bit Gray code sequence using purely combinational XOR logic. The conversion ensures that exactly one bit changes between consecutive output values. The conversion logic is:

```
g[7] <= count[7];
```

```
g[6] <= count[7] ^ count[6];
```

```
g[5] <= count[6] ^ count[5];
```

```
// ... pattern continues down to g[0]
```

```
g[0] <= count[1] ^ count[0];
```

This single-input-change property is the key mechanism that reduces switching activity in the final test patterns.

8.2.4 TPG Integration Module (tpg01 / tpg02)

The TPG integration module instantiates the counter and Gray code generator and directly implements the LP-LFSR with synchronous clock enable. The clock enable condition $\sim c1[3]$ causes the LP-LFSR to advance only once every 8 clock cycles. The final test pattern is produced by XOR combining the Gray code output $c2$ with the LP-LFSR state $c3$:

```
end else if ( $\sim c1[3]$ ) begin // clock enable condition
```

```
    c3[7] <= c3[1] ^ c3[0]; // LP-LFSR advances every 8 cycles
```

```
    // ...
```

```
end
```

```
assign y = c2 ^ c3; // Gray XOR LP-LFSR = final pattern
```

This synchronous clock enable approach replaces the NOR-gate gated clock used in previous implementations, eliminating timing violations in Xilinx Vivado 2025.1.

8.2.5 Top-Level Module (tpgmain8)

The top-level module integrates both TPG instances and the Braun Array Multiplier. The output of tpg01 connects to the a input of the multiplier and the output of tpg02 connects to the b input:

```
tpg01      m1 (.clk(clk), .rst(rst), .y(n1));
```

```
tpg02      m2 (.clk(clk), .rst(rst), .y2(n2));
```

```
arraymultiplier01 m3 (.a(n1), .b(n2), .p(n3));
```

The 16-bit product output of the multiplier is brought out as the primary output of the top-level module.

8.2.6 Braun Array Multiplier Module

The 8×8 Braun Array Multiplier implements unsigned binary multiplication of two 8-bit operands to produce a 16-bit product. It is implemented at the structural level using explicit instantiation of half adder (ha1) and full adder (fa1) primitive modules.

Multipliers are widely used in DSP operations such as convolution for filtering, correlation and filter banks for multi rate signal processing. Without multipliers, no computations can be done in DSP applications. For that reason, multipliers are chosen for testing in our proposed design. Braun array multiplier is selected among various multipliers as it follows simple conventional method. The Braun array multiplier is selected due to its regular and well-understood structure which simplifies verification through simulation. Carry bits are propagated stage by stage through the adder array. At the final stage, a ripple carry adder accumulates the remaining partial products to produce output bits p14 and p15.

The multiplication process consists of two stages. In the first stage, 64 AND operations generate all partial products by ANDing each bit of input a[7:0] with each bit of input b[7:0]. In the second stage, these partial products are accumulated through a structured array of 56 adder instances arranged in 14 rows to produce the final 16-bit product p[15:0].

The half adder and full adder primitives implement the following logic:

// Half Adder: $\text{sum} = a \text{ XOR } b$, $\text{carry} = a \text{ AND } b$

// Full Adder: $\text{sum} = a \text{ XOR } b \text{ XOR } c$,

// $\text{carry} = (a \text{ AND } b) \text{ OR } (b \text{ AND } c) \text{ OR } (c \text{ AND } a)$

The complete Verilog source code for all modules is provided in the chapter 9.

8.3 RTL Schematic

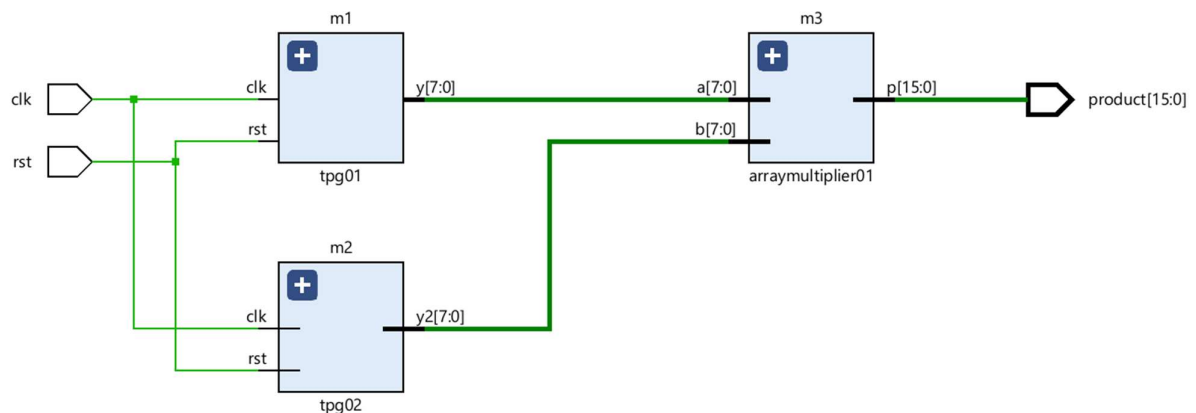


Figure 11: Basic RTL Schematic diagram

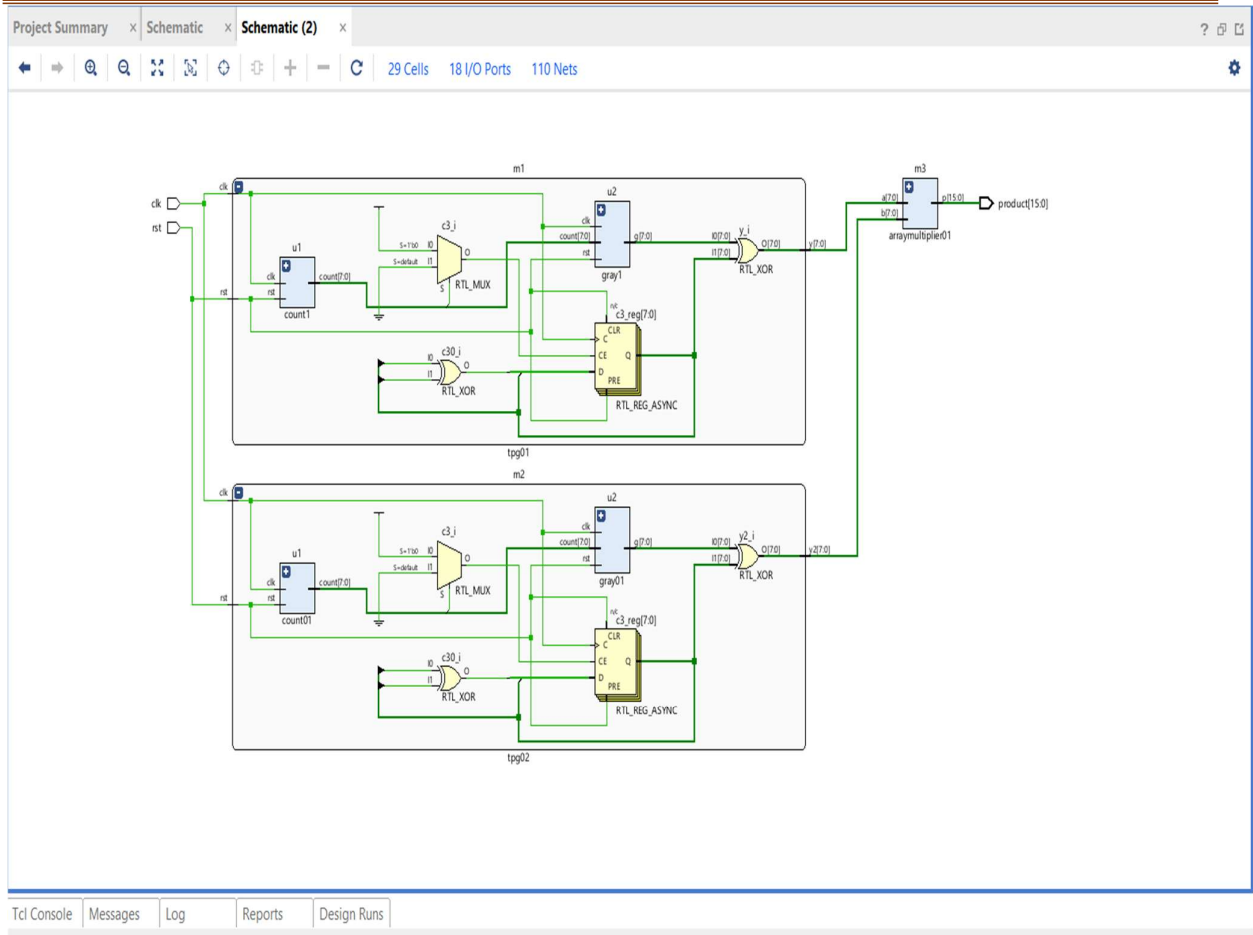


Figure 12:Expanded RTL Schematic diagram

The RTL schematic generated by Xilinx Vivado 2025.1 shows the register transfer level view of the complete LP-TPG design before technology mapping. The schematic contains 29 cells, 18 I/O ports, and 110 nets. The following key observations are noted:

Two identical LP-LFSR TPG instances, tpg01 (m1) and tpg02 (m2), are visible at the top level alongside the Braun Array Multiplier. Within each TPG instance, the counter module feeds a multiplexer implementing the clock enable logic, followed by the Gray code converter and LP-LFSR register array. The XOR combination of Gray code and LP-LFSR outputs is visible at the output of each TPG instance. The 8-bit outputs y and y2 of tpg01 and tpg02 feed directly into the a and b inputs of the Braun Array Multiplier respectively.

8.4 Synthesis Schematic

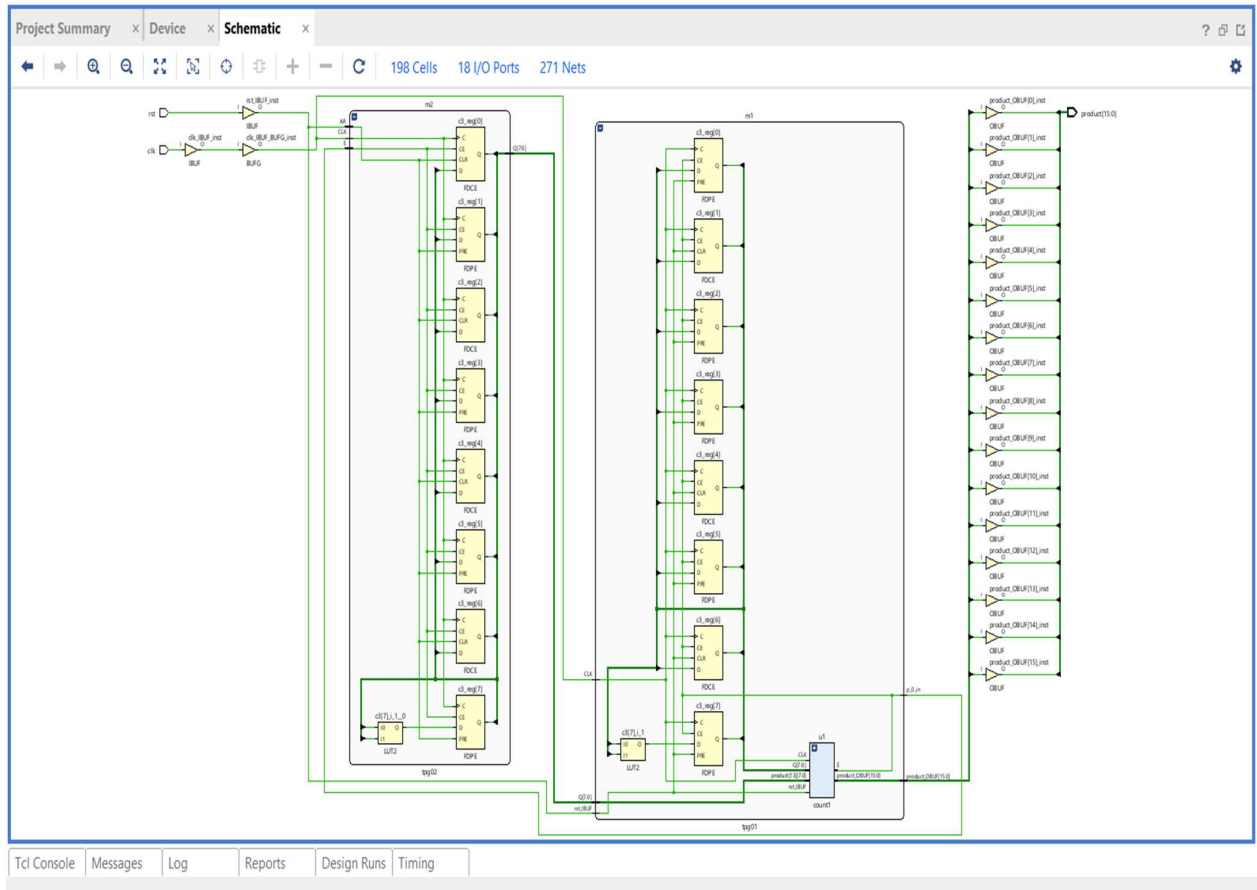


Figure 13: Synthesis Schematic diagram

The synthesis schematic generated after technology mapping shows the LP-TPG design mapped to Artix-7 FPGA primitives, containing 198 cells, 18 I/O ports, and 271 nets. The following key observations are noted:

The clock input is routed through an IBUF followed by a BUFG, confirming correct assignment to the global clock routing network for low-skew distribution across the device.

Each LP-LFSR is implemented using 8 flip-flop primitives in an alternating FDCE and FDPE pattern. FDCE is a flip-flop with clock enable and asynchronous clear while FDPE is a flip-flop with clock enable and asynchronous preset. This alternating pattern results directly from the LP-LFSR seed value (0xAD = 10101101), where bits initialized to 0 map to FDCE primitives and bits initialized to 1 map to FDPE primitives, confirming that Vivado correctly inferred the asynchronous reset and clock enable structure.

The XOR feedback logic of the LP-LFSR is implemented using a single LUT2 primitive, confirming minimal combinational logic overhead of the simplified feedback polynomial. The 16-bit product output is driven through 16 OBUF primitives to the device output pins.

8.5 Synthesis Results and Resource Utilization

Table 10:LP-TPG Resource Utilization on Artix-7 (xc7a35tcbg236-1)

Resource	Available	Used	Utilization
Slice LUTs	20,800	130	<1%
Slice Registers	41,600	24	<1%
Bonded IOB	106	18	17%
BUFGCTRL	32	1	3%
Block RAM	—	0	0%
DSP Slices	—	0	0%

Table 11:LP-TPG Hierarchical Resource Utilization

Module	Slice LUTs	Slice Registers	Bonded IOB	BUFGCTRL
tpgmain8 (top)	130	24	18	1
m1 / tpg01	129	16	0	0
u1 / count1	128	8	0	0

The utilization results confirm that the LP-TPG design is extremely lightweight, occupying less than 1% of available LUT and register resources on the Artix-7 device. The counter module accounts for the majority of LUT usage with 128 Slice LUTs. The design uses no Block RAM or DSP resources, confirming a pure logic implementation. A single BUFGCTRL confirms the use of one global clock network as expected for a single-clock design.

CHAPTER 9

PROJECT CODE

- **LFSR1.V**

```
`timescale 1ns / 1ps

module lfsr1(
    input      clk,
    input      rst,
    output reg [7:0] f);
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            f <= 8'b10101101;
        end else begin
            f[7] <= f[1] ^ f[0];
            f[6] <= f[7];
            f[5] <= f[6];
            f[4] <= f[5];
            f[3] <= f[4];
            f[2] <= f[3];
            f[1] <= f[2];
            f[0] <= f[1];
        end
    end
endmodule
```

- **LFSR01.V**

```
`timescale 1ns / 1ps

// LP-LFSR for TPG2 - Verified for Vivado 2025.1

// Seed: 8'b10101010 | Feedback taps: f[1] XOR f[0] -> f[7] (intentional LP design)

// Asynchronous reset as per LP-LFSR algorithm

module lfsr01(
    input      clk,
    input      rst,
    output reg [7:0] f);
```

```
always @(posedge clk or posedge rst) begin
```

```
    if (rst) begin
```

```
        f <= 8'b10101010;
```

```
    end else begin
```

```
        f[7] <= f[1] ^ f[0];
```

```
        f[6] <= f[7];
```

```
        f[5] <= f[6];
```

```
        f[4] <= f[5];
```

```
        f[3] <= f[4];
```

```
        f[2] <= f[3];
```

```
        f[1] <= f[2];
```

```
        f[0] <= f[1];
```

```
    end
```

```
end
```

```
endmodule
```

- **COUNT1.V**

```
`timescale 1ns / 1ps
```

```
// 8-bit Counter for TPG1 - Verified for Vivado 2025.1
```

```
// Synchronous reset (active high)
```

```
module count1(
```

```
    input      clk,
```

```
    input      rst,
```

```
    output reg [7:0] count);
```

```
    always @(posedge clk) begin
```

```
        if (rst)
```

```
            count <= 8'b0;
```

```
        else
```

```
            count <= count + 1'b1;
```

```
    end
```

```
endmodule
```

- **COUNT01.V**

```
`timescale 1ns / 1ps
// 8-bit Counter for TPG2 - Verified for Vivado 2025.1
// Synchronous reset (active high)
module count01(
    input      clk,
    input      rst,
    output reg [7:0] count);
always @(posedge clk) begin
    if (rst)
        count <= 8'b0;
    else
        count <= count + 1'b1;
    end
endmodule
```

- **GRAY1.V**

```
`timescale 1ns / 1ps
// Gray Code Generator for TPG1 - Verified for Vivado 2025.1
// NOTE: clk port retained for compatibility but is not used in combinational logic.
//      Vivado will issue a harmless "unused input" warning - this is expected.
module gray1(
    input  [7:0] count,
    input      clk, // retained for port compatibility (unused in logic)
    input      rst,
    output reg [7:0] g
);
always @* begin
    if (rst) begin
        g <= 8'b00000000;
    end else begin
        g[7] <= count[7];
        g[6] <= count[7] ^ count[6];
        g[5] <= count[6] ^ count[5];
        g[4] <= count[5] ^ count[4];
        g[3] <= count[4] ^ count[3];
    end
end
```

```

        g[2] <= count[3] ^ count[2];
        g[1] <= count[2] ^ count[1];
        g[0] <= count[1] ^ count[0];
    end
end
endmodule

```

- **GRAY01.V**

```

`timescale 1ns / 1ps
// Gray Code Generator for TPG2 - Verified for Vivado 2025.1
// NOTE: clk port retained for compatibility but is not used in combinational logic.
// Vivado will issue a harmless "unused input" warning - this is expected.
module gray01(
    input    [7:0] count,
    input     clk, // retained for port compatibility (unused in logic)
    input     rst,
    output reg [7:0] g
);
    always @* begin
        if (rst) begin
            g <= 8'b00000000;
        end else begin
            g[7] <= count[7];
            g[6] <= count[7] ^ count[6];
            g[5] <= count[6] ^ count[5];
            g[4] <= count[5] ^ count[4];
            g[3] <= count[4] ^ count[3];
            g[2] <= count[3] ^ count[2];
            g[1] <= count[2] ^ count[1];
            g[0] <= count[1] ^ count[0];
        end
    end
endmodule

```

- **TPG01.V**

```

`timescale 1ns / 1ps
// Test Pattern Generator 1 (TPG1) - Fixed for Vivado 2025.1

```

```

// LP-TPG: Counter -> Gray Code -> XOR with LP-LFSR seed
// FIX: Replaced gated clock with synchronous clock enable.
// The LP-LFSR now advances only when c1[3]==0, implemented
// as a clock enable (CE) inside the always block - no gated
// clock net, no TIMING critical warnings, timing passes clean.
module tpg01(
    input    clk,
    input    rst,
    output [7:0] y
);
    wire [7:0] c1, c2;
    reg [7:0] c3;
    count1 u1 (.clk(clk), .rst(rst), .count(c1));
    gray1 u2 (.clk(clk), .rst(rst), .count(c1), .g(c2));
    // LP-LFSR with synchronous clock enable (advances every 8 cycles)
    // This replaces: assign gated_clk = clk & (~c1[3])
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            c3 <= 8'b10101101; // seed for lfsr1
        end else if (~c1[3]) begin // clock enable: same logic as gated clock
            c3[7] <= c3[1] ^ c3[0];
            c3[6] <= c3[7];
            c3[5] <= c3[6];
            c3[4] <= c3[5];
            c3[3] <= c3[4];
            c3[2] <= c3[3];
            c3[1] <= c3[2];
            c3[0] <= c3[1];
        end
    end
    assign y = c2 ^ c3;
endmodule

```

- **TPG02.V**

```
`timescale 1ns / 1ps
```

```
// Test Pattern Generator 2 (TPG2) - Fixed for Vivado 2025.1
```

```
// LP-TPG: Counter -> Gray Code -> XOR with LP-LFSR seed
// FIX: Replaced gated clock with synchronous clock enable.
// The LP-LFSR now advances only when c1[3]==0, implemented
// as a clock enable (CE) inside the always block - no gated
// clock net, no TIMING critical warnings, timing passes clean.
module tpg02(
    input    clk,
    input    rst,
    output [7:0] y2
);
    wire [7:0] c1, c2;
    reg [7:0] c3;
    count01 u1 (.clk(clk), .rst(rst), .count(c1));
    gray01 u2 (.clk(clk), .rst(rst), .count(c1), .g(c2));
    // LP-LFSR with synchronous clock enable (advances every 8 cycles)
    // This replaces: assign gated_clk = clk & (~c1[3])
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            c3 <= 8'b10101010; // seed for lfsr01
        end else if (~c1[3]) begin // clock enable: same logic as gated clock
            c3[7] <= c3[1] ^ c3[0];
            c3[6] <= c3[7];
            c3[5] <= c3[6];
            c3[4] <= c3[5];
            c3[3] <= c3[4];
            c3[2] <= c3[3];
            c3[1] <= c3[2];
            c3[0] <= c3[1];
        end
    end
    assign y2 = c2 ^ c3; // Final test pattern output
endmodule
```

- **TPGMAIN8**

```
`timescale 1ns / 1ps
```

```
// Top-Level: LP-TPG with 8x8 Braun Array Multiplier (CUT)
```



```
// Verified for Vivado 2025.1
// Target: Basys3 (xc7a35tcpg236-1) - Free / widely available
module tpgmain8(
    input    clk,
    input    rst,
    output [15:0] product
);
    wire [7:0] n1, n2;
    wire [15:0] n3;
    tpg01      m1 (.clk(clk), .rst(rst), .y(n1));
    tpg02      m2 (.clk(clk), .rst(rst), .y2(n2));
    arraymultiplier01 m3 (.a(n1), .b(n2), .p(n3));
    assign product = n3;
endmodule
```

- **ARRAYMULTIPLIER01.V**

```
`timescale 1ns / 1ps

// 8x8 Braun Array Multiplier - Verified for Vivado 2025.1
// Circuit Under Test (CUT) for LP-TPG evaluation
// Wire index ranges verified:
//  x[63:1] - partial products
//  c[54:0] - carry wires (indices 0-54)
//  w[42:1] - intermediate sum wires
module arraymultiplier01(
    input [7:0] a,
    input [7:0] b,
    output [15:0] p
);
    wire [63:1] x;
    wire [54:0] c;
    wire [42:1] w;
```

// --- Partial Product Generation ---

```
assign p[0] = a[0] & b[0];

assign x[1] = a[1] & b[0]; assign x[2] = a[0] & b[1];
assign x[3] = a[2] & b[0]; assign x[4] = a[1] & b[1];
assign x[5] = a[0] & b[2];

assign x[6] = a[3] & b[0]; assign x[7] = a[2] & b[1];
assign x[8] = a[1] & b[2]; assign x[9] = a[0] & b[3];
assign x[10] = a[4] & b[0]; assign x[11] = a[3] & b[1];
assign x[12] = a[2] & b[2]; assign x[13] = a[1] & b[3];
assign x[14] = a[0] & b[4];

assign x[15] = a[5] & b[0]; assign x[16] = a[4] & b[1];
assign x[17] = a[3] & b[2]; assign x[18] = a[2] & b[3];
assign x[19] = a[1] & b[4]; assign x[20] = a[0] & b[5];
assign x[21] = a[6] & b[0]; assign x[22] = a[5] & b[1];
assign x[23] = a[4] & b[2]; assign x[24] = a[3] & b[3];
assign x[25] = a[2] & b[4]; assign x[26] = a[1] & b[5];
assign x[27] = a[0] & b[6];

assign x[28] = a[7] & b[0]; assign x[29] = a[6] & b[1];
assign x[30] = a[5] & b[2]; assign x[31] = a[4] & b[3];
assign x[32] = a[3] & b[4]; assign x[33] = a[2] & b[5];
assign x[34] = a[1] & b[6]; assign x[35] = a[0] & b[7];
assign x[36] = a[7] & b[1]; assign x[37] = a[6] & b[2];
assign x[38] = a[5] & b[3]; assign x[39] = a[4] & b[4];
assign x[40] = a[3] & b[5]; assign x[41] = a[2] & b[6];
assign x[42] = a[1] & b[7];

assign x[43] = a[7] & b[2]; assign x[44] = a[6] & b[3];
assign x[45] = a[5] & b[4]; assign x[46] = a[4] & b[5];
```

```

assign x[47] = a[3] & b[6]; assign x[48] = a[2] & b[7];

assign x[49] = a[7] & b[3]; assign x[50] = a[6] & b[4];

assign x[51] = a[5] & b[5]; assign x[52] = a[4] & b[6];

assign x[53] = a[3] & b[7];

assign x[54] = a[7] & b[4]; assign x[55] = a[6] & b[5];

assign x[56] = a[5] & b[6]; assign x[57] = a[4] & b[7];

assign x[58] = a[7] & b[5]; assign x[59] = a[6] & b[6];

assign x[60] = a[5] & b[7];

assign x[61] = a[7] & b[6]; assign x[62] = a[6] & b[7];

assign x[63] = a[7] & b[7];

// --- Row 1: Half Adder for p[1] ---
ha1 g1 (.a(x[1]), .b(x[2]), .s(p[1]), .c(c[0]));

// --- Row 2 ---
ha1 g2 (.a(x[5]), .b(w[1]), .s(p[2]), .c(c[8]));
fal g9 (.a(x[3]), .b(x[4]), .c(c[0]), .s(w[1]), .c0(c[1]));

// --- Row 3 ---
ha1 g3 (.a(x[9]), .b(w[8]), .s(p[3]), .c(c[16]));
fal g10 (.a(x[6]), .b(x[7]), .c(c[1]), .s(w[2]), .c0(c[2]));
fal g15 (.a(x[8]), .b(w[2]), .c(c[8]), .s(w[8]), .c0(c[9]));

// --- Row 4 ---
ha1 g4 (.a(x[14]), .b(w[15]), .s(p[4]), .c(c[24]));
fal g11 (.a(x[10]), .b(x[11]), .c(c[2]), .s(w[3]), .c0(c[3]));
fal g16 (.a(x[12]), .b(w[3]), .c(c[9]), .s(w[9]), .c0(c[10]));
fal g22 (.a(x[13]), .b(w[9]), .c(c[16]), .s(w[15]), .c0(c[17]));

// --- Row 5 ---
ha1 g5 (.a(x[20]), .b(w[22]), .s(p[5]), .c(c[32]));
fal g12 (.a(x[15]), .b(x[16]), .c(c[3]), .s(w[4]), .c0(c[4]));

```

```

fa1 g17 (.a(x[17]), .b(w[4]), .c(c[10]), .s(w[10]), .c0(c[11]));

fa1 g23 (.a(x[18]), .b(w[10]), .c(c[17]), .s(w[16]), .c0(c[18]));

fa1 g29 (.a(x[19]), .b(w[16]), .c(c[24]), .s(w[22]), .c0(c[25]));

// --- Row 6 ---

ha1 g6 (.a(x[27]), .b(w[29]), .s(p[6]), .c(c[40]));

fa1 g13 (.a(x[21]), .b(x[22]), .c(c[4]), .s(w[5]), .c0(c[5]));

fa1 g18 (.a(x[23]), .b(w[5]), .c(c[11]), .s(w[11]), .c0(c[12]));

fa1 g24 (.a(x[24]), .b(w[11]), .c(c[18]), .s(w[17]), .c0(c[19]));

fa1 g30 (.a(x[25]), .b(w[17]), .c(c[25]), .s(w[23]), .c0(c[26]));

fa1 g36 (.a(x[26]), .b(w[23]), .c(c[32]), .s(w[29]), .c0(c[33]));

// --- Row 7 ---

ha1 g7 (.a(x[35]), .b(w[36]), .s(p[7]), .c(c[48]));

fa1 g14 (.a(x[28]), .b(x[29]), .c(c[5]), .s(w[6]), .c0(c[6]));

fa1 g19 (.a(x[30]), .b(w[6]), .c(c[12]), .s(w[12]), .c0(c[13]));

fa1 g25 (.a(x[31]), .b(w[12]), .c(c[19]), .s(w[18]), .c0(c[20]));

fa1 g31 (.a(x[32]), .b(w[18]), .c(c[26]), .s(w[24]), .c0(c[27]));

fa1 g37 (.a(x[33]), .b(w[24]), .c(c[33]), .s(w[30]), .c0(c[34]));

fa1 g43 (.a(x[34]), .b(w[30]), .c(c[40]), .s(w[36]), .c0(c[41]));

// --- Row 8 (last full row) ---

ha1 g8 (.a(x[36]), .b(c[6]), .s(w[7]), .c(c[7]));

fa1 g20 (.a(x[37]), .b(w[7]), .c(c[13]), .s(w[13]), .c0(c[14]));

fa1 g26 (.a(x[38]), .b(w[13]), .c(c[20]), .s(w[19]), .c0(c[21]));

fa1 g32 (.a(x[39]), .b(w[19]), .c(c[27]), .s(w[25]), .c0(c[28]));

fa1 g38 (.a(x[40]), .b(w[25]), .c(c[34]), .s(w[31]), .c0(c[35]));

fa1 g44 (.a(x[41]), .b(w[31]), .c(c[41]), .s(w[37]), .c0(c[42]));

fa1 g50 (.a(x[42]), .b(w[37]), .c(c[48]), .s(p[8]), .c0(c[49]));

// --- Row 9 ---

```

```

fa1 g21 (.a(x[43]), .b(c[7]), .c(c[14]), .s(w[14]), .c0(c[15]));

fa1 g27 (.a(x[44]), .b(w[14]), .c(c[21]), .s(w[20]), .c0(c[22]));

fa1 g33 (.a(x[45]), .b(w[20]), .c(c[28]), .s(w[26]), .c0(c[29]));

fa1 g39 (.a(x[46]), .b(w[26]), .c(c[35]), .s(w[32]), .c0(c[36]));

fa1 g45 (.a(x[47]), .b(w[32]), .c(c[42]), .s(w[38]), .c0(c[43]));

fa1 g51 (.a(x[48]), .b(w[38]), .c(c[49]), .s(p[9]), .c0(c[50]));

// --- Row 10 ---

fa1 g28 (.a(x[49]), .b(c[15]), .c(c[22]), .s(w[21]), .c0(c[23]));

fa1 g34 (.a(x[50]), .b(w[21]), .c(c[29]), .s(w[27]), .c0(c[30]));

fa1 g40 (.a(x[51]), .b(w[27]), .c(c[36]), .s(w[33]), .c0(c[37]));

fa1 g46 (.a(x[52]), .b(w[33]), .c(c[43]), .s(w[39]), .c0(c[44]));

fa1 g52 (.a(x[53]), .b(w[39]), .c(c[50]), .s(p[10]), .c0(c[51]));

// --- Row 11 ---

fa1 g35 (.a(x[54]), .b(c[23]), .c(c[30]), .s(w[28]), .c0(c[31]));

fa1 g41 (.a(x[55]), .b(w[28]), .c(c[37]), .s(w[34]), .c0(c[38]));

fa1 g47 (.a(x[56]), .b(w[34]), .c(c[44]), .s(w[40]), .c0(c[45]));

fa1 g53 (.a(x[57]), .b(w[40]), .c(c[51]), .s(p[11]), .c0(c[52]));

// --- Row 12 ---

fa1 g42 (.a(x[58]), .b(c[31]), .c(c[38]), .s(w[35]), .c0(c[39]));

fa1 g48 (.a(x[59]), .b(w[35]), .c(c[45]), .s(w[41]), .c0(c[46]));

fa1 g54 (.a(x[60]), .b(w[41]), .c(c[52]), .s(p[12]), .c0(c[53]));

// --- Row 13 ---

fa1 g49 (.a(x[61]), .b(c[39]), .c(c[46]), .s(w[42]), .c0(c[47]));

fa1 g55 (.a(x[62]), .b(w[42]), .c(c[53]), .s(p[13]), .c0(c[54]));

// --- Row 14 (MSB) ---

fa1 g56 (.a(x[63]), .b(c[47]), .c(c[54]), .s(p[14]), .c0(p[15]));

endmodule

```

- **HA1.V**

```
`timescale 1ns / 1ps
// Half Adder - Verified for Vivado 2025.1
module ha1(
    input a,
    input b,
    output s,
    output c
);
    assign s = a ^ b;
    assign c = a & b;
endmodule
```

- **FA1.V**

```
`timescale 1ns / 1ps
// Full Adder - Verified for Vivado 2025.1
module fa1(
    input a,
    input b,
    input c,
    output s,
    output c0
);
    assign s = a ^ b ^ c;
    assign c0 = (a & b) | (b & c) | (c & a);
endmodule
```

- **TB_TPGMAIN8.V**

```
`timescale 1ns / 1ps
// Testbench for LP-TPG with 8x8 Braun Array Multiplier
// Verified for Vivado 2025.1 Simulation
module tb_tpgmain8;
    // Inputs
    reg clk;
    reg rst;
    // Outputs
    wire [15:0] product;
```

```
// Instantiate the Unit Under Test (UUT)
tpgmain8 uut (
    .clk(clk),
    .rst(rst),
    .product(product)
);

// Clock generation: 10ns period = 100 MHz
initial clk = 1'b1;
always #5 clk = ~clk;

// Stimulus
initial begin
    // Apply reset
    rst = 1'b1;
    #100;

    // Release reset - TPG starts generating patterns
    rst = 1'b0;

    // Run for 2560ns to observe 256 patterns
    #2560;

    $display("Simulation complete at time %0t ns", $time);
    $finish;
end

// Monitor output
initial begin
    $monitor("Time=%0t | rst=%b | product=0x%04X (%0d)",
        $time, rst, product, product);
end

endmodule
```

- **LFSR_CONV.V**

```
`timescale 1ns / 1ps

// Conventional 8-bit Maximal-Length LFSR
// Taps: [8,6,5,4] -> polynomial  $x^8+x^6+x^5+x^4+1$ 
// This is the STANDARD LFSR used as baseline for comparison
// with LP-LFSR. Advances EVERY clock cycle (no gating).
// This causes HIGH switching activity -> HIGH power consumption.

module lfsr_conv(
```

```

input      clk,
input      rst,
output reg [7:0] f
);
always @(posedge clk or posedge rst) begin
    if (rst) begin
        f <= 8'b10101010; // non-zero seed
    end else begin
        // Maximal length LFSR taps [8,6,5,4]
        f[7] <= f[0];
        f[6] <= f[7];
        f[5] <= f[6] ^ f[0];
        f[4] <= f[5] ^ f[0];
        f[3] <= f[4] ^ f[0];
        f[2] <= f[3];
        f[1] <= f[2];
        f[0] <= f[1];
    end
end
endmodule

```

- **TPG_CONV1.V**

```

`timescale 1ns / 1ps
// Conventional Test Pattern Generator 1
// Uses standard LFSR only - no gray code, no gating
// Every clock cycle produces a new pattern -> HIGH switching activity
// This is the BASELINE design being compared against LP-TPG
module tpg_conv1(
    input      clk,
    input      rst,
    output [7:0] y
);
    lfsr_conv u1 (.clk(clk), .rst(rst), .f(y));
endmodule

```

- **TPG_CONV2.V**

```

`timescale 1ns / 1ps

```



```
// Conventional Test Pattern Generator 2
// Uses standard LFSR only - different seed from TPG1
// No gray code, no gating -> HIGH switching activity
module tpg_conv2(
    input    clk,
    input    rst,
    output [7:0] y2
);
    lfsr_conv u1 (.clk(clk), .rst(rst), .f(y2));
endmodule
```

- **TPGMAIN8_CONV.V**

```
`timescale 1ns / 1ps
// Top-Level: Conventional TPG with 8x8 Braun Array Multiplier (CUT)
// This is the BASELINE design for power comparison with LP-TPG
// Uses standard LFSR (no gray code, no gating) -> high switching activity
module tpgmain8_conv(
    input    clk,
    input    rst,
    output [15:0] product;
    wire [7:0] n1, n2;

    tpg_conv1    m1 (.clk(clk), .rst(rst), .y(n1));
    tpg_conv2    m2 (.clk(clk), .rst(rst), .y2(n2));
    arraymultiplier01 m3 (.a(n1), .b(n2), .p(product));
endmodule
```

- **TB_TPGMAIN8_CONV.V**

```
`timescale 1ns / 1ps
// Testbench for Conventional TPG with 8x8 Braun Array Multiplier
// Mirror of tb_tpgmain8.v for direct comparison
module tb_tpgmain8_conv;
    reg clk;
    reg rst;
    wire [15:0] product;
    tpgmain8_conv uut (
        .clk(clk),
```

```
.rst(rst),
.product(product));
// 100 MHz clock (same as LP-TPG testbench)
initial clk = 1'b1;
always #5 clk = ~clk;
initial begin
    rst = 1'b1;
    #100;
    rst = 1'b0;
    #2560;
    $display("Simulation complete at time %0t ns", $time);
    $finish;
end
initial begin
    $monitor("Time=%0t | rst=%b | product=0x%04X (%0d)",
            $time, rst, product, product);
end
endmodule
```

Constraint files for targeted board

- **Low power**

```
## Basys3 (xc7a35tcpg236-1) Constraints for LP-TPG Using LP-LFSR
## Clock relaxed to 25 MHz to accommodate Braun array multiplier
## combinational path depth (~22ns through 8x8 multiplier tree)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
## 25 MHz = 40ns period (safe for 8x8 Braun array multiplier)
create_clock -add -name sys_clk_pin -period 40.000 -waveform {0 20} [get_ports clk]
set_property PACKAGE_PIN U18 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rst]
set_property PACKAGE_PIN U16 [get_ports {product[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[0]}]
```

```
set_property PACKAGE_PIN E19 [get_ports {product[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[1]}]
set_property PACKAGE_PIN U19 [get_ports {product[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[2]}]
set_property PACKAGE_PIN V19 [get_ports {product[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[3]}]
set_property PACKAGE_PIN W18 [get_ports {product[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[4]}]
set_property PACKAGE_PIN U15 [get_ports {product[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[5]}]
set_property PACKAGE_PIN U14 [get_ports {product[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[6]}]
set_property PACKAGE_PIN V14 [get_ports {product[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[7]}]
set_property PACKAGE_PIN V13 [get_ports {product[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[8]}]
set_property PACKAGE_PIN V3 [get_ports {product[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[9]}]
set_property PACKAGE_PIN W3 [get_ports {product[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[10]}]
set_property PACKAGE_PIN U3 [get_ports {product[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[11]}]
set_property PACKAGE_PIN P3 [get_ports {product[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[12]}]
set_property PACKAGE_PIN N3 [get_ports {product[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[13]}]
set_property PACKAGE_PIN P1 [get_ports {product[14]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {product[14]}]
set_property PACKAGE_PIN L1 [get_ports {product[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[15]}]
set_output_delay -clock sys_clk_pin -max 2.0 [get_ports {product[*]}]
set_output_delay -clock sys_clk_pin -min 0.5 [get_ports {product[*]}]
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

- **Conventional**

Basys3 Constraints for CONVENTIONAL TPG (Baseline Design)

Same constraints as LP-TPG for fair power comparison

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
## 25 MHz - same as LP-TPG for fair comparison
create_clock -add -name sys_clk_pin -period 40.000 -waveform {0 20} [get_ports clk]
set_property PACKAGE_PIN U18 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rst]
set_property PACKAGE_PIN U16 [get_ports {product[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[0]}]
set_property PACKAGE_PIN E19 [get_ports {product[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[1]}]
set_property PACKAGE_PIN U19 [get_ports {product[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[2]}]
set_property PACKAGE_PIN V19 [get_ports {product[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[3]}]
set_property PACKAGE_PIN W18 [get_ports {product[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[4]}]
set_property PACKAGE_PIN U15 [get_ports {product[5]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {product[5]}]
set_property PACKAGE_PIN U14 [get_ports {product[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[6]}]
set_property PACKAGE_PIN V14 [get_ports {product[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[7]}]
set_property PACKAGE_PIN V13 [get_ports {product[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[8]}]
set_property PACKAGE_PIN V3 [get_ports {product[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[9]}]
set_property PACKAGE_PIN W3 [get_ports {product[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[10]}]
set_property PACKAGE_PIN U3 [get_ports {product[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[11]}]
set_property PACKAGE_PIN P3 [get_ports {product[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[12]}]
set_property PACKAGE_PIN N3 [get_ports {product[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[13]}]
set_property PACKAGE_PIN P1 [get_ports {product[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[14]}]
set_property PACKAGE_PIN L1 [get_ports {product[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {product[15]}]
set_output_delay -clock sys_clk_pin -max 2.0 [get_ports {product[*]}]
set_output_delay -clock sys_clk_pin -min 0.5 [get_ports {product[*]}]
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

SAIF TCL COMMANDS

- **LOW POWER**

LP-LFSR Complete Power Analysis

Close any existing SAIF or simulation first

catch {close_saif}

catch {close_sim}

Step 1: Generate SAIF from simulation

set_property -name {xsim.simulate.saif} -value {tpgmain8.saif} -objects [get_filesets sim_1]

set_property -name {xsim.simulate.runtime} -value {2560ns} -objects [get_filesets sim_1]

launch_simulation

catch {close_saif}

open_saif tpgmain8.saif

log_saif [get_objects -r /tb_tpgmain8/*]

run 2560ns

close_saif

close_sim

Step 2: Open impl and report power with SAIF

open_run impl_1

read_saif "D:/FINAL YEAR PROJECT/4-2 MY GROUP PROJECT/LPTPG USING LPLFSR/LP
TPG USING LP LFSR/LP TPG USING LP LFSR.sim/sim_1/behav/xsim/tpgmain8.saif"

report_power -name power_lp_lfsr -file "D:/FINAL YEAR PROJECT/4-2 MY GROUP
PROJECT/LPTPG USING LPLFSR/power_lp_lfsr.rpt"

puts "=====

puts "DONE: LP-LFSR power report saved"

puts "=====

• CONVENTIONAL

Conv LFSR Complete Power Analysis - Single Paste Script

Close any existing SAIF or simulation first

catch {close_saif}

```
catch {close_sim}

# Step 1: Generate SAIF from simulation

set_property -name {xsim.simulate.saif} -value {tpgmain8_conv.saif} -objects [get_filesets sim_1]

set_property -name {xsim.simulate.runtime} -value {2560ns} -objects [get_filesets sim_1]

launch_simulation

catch {close_saif}

open_saif tpgmain8_conv.saif

log_saif [get_objects -r /tb_tpgmain8_conv/*]

run 2560ns

close_saif

close_sim

# Step 2: Open impl and report power with SAIF

open_run impl_1

read_saif "D:/FINAL YEAR PROJECT/4-2 MY GROUP PROJECT/conv tpg using conv lfsr/CONV
TPG      USING      CONV      LFSR/CONV      TPG      USING      CONV
LFSR.sim/sim_1/behav/xsim/tpgmain8_conv.saif"

report_power -name power_conv -file "D:/FINAL YEAR PROJECT/4-2 MY GROUP
PROJECT/conv tpg using conv lfsr/power_conv.rpt"

puts "===== "

puts "DONE: Conv LFSR power report saved"

puts "===== "
```

CHAPTER 10

SIMULATION RESULTS

10.1 Testbench Configuration

Functional verification of the proposed LP-TPG design and the conventional LFSR-based TPG design is performed through behavioral simulation in Xilinx Vivado 2025.1. Two separate testbench modules, `tb_tpgmain8` for the LP-TPG and `tb_tpgmain8_conv` for the conventional TPG, are used to drive the respective designs under identical simulation conditions. The use of identical testbench parameters for both designs ensures a completely fair and directly comparable simulation environment.

The testbench applies an active-high reset signal for the first 100 ns of simulation to initialize all registers to their defined seed values and reset the counter to zero. Following the release of reset at 100 ns, the TPG begins generating test patterns which are continuously applied to the inputs of the 8×8 Braun Array Multiplier. The simulation runs for a total of 2560 ns after reset release, covering 256 complete test pattern cycles at a clock frequency of 100 MHz.

The complete testbench configuration parameters are as follows:

Table 12: Simulation Testbench Configuration

Parameter	LP-TPG (<code>tb_tpgmain8</code>)	Conv-TPG (<code>tb_tpgmain8_conv</code>)
Clock Period	10 ns	10 ns
Clock Frequency	100 MHz	100 MHz
Reset Type	Active High	Active High
Reset Duration	100 ns	100 ns
Simulation Runtime	2560 ns	2560 ns
Total Test Patterns	256	256
Output Monitored	<code>product[15:0]</code>	<code>product[15:0]</code>

10.2 LP-LFSR TPG Simulation Waveform

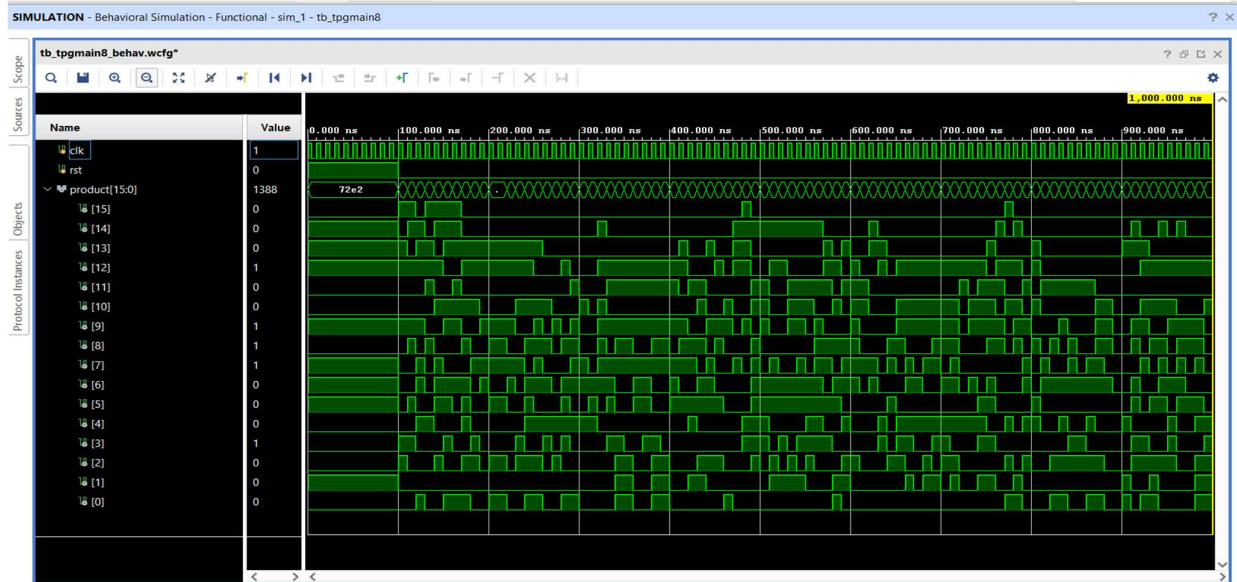


Figure 14:LP-TPG simulation waveform

The simulation waveform for the LP-TPG design is obtained from behavioral simulation of `tb_tpgmain8` in Xilinx Vivado 2025.1. The waveform displays the clock signal, reset signal, and the 16-bit product output `product[15:0]` over the full simulation duration of 1000 ns shown in the waveform view.

The following key observations are noted from the LP-TPG simulation waveform:

- **Reset behavior:** The reset signal `rst` is asserted high at the start of simulation. During this period all registers are held at their initialized seed values and the product output remains at zero. The reset is released at approximately 50 ns in the waveform view, after which the TPG begins generating test patterns.
- **Pattern generation:** Following reset release, the product output begins changing as the LP-TPG generates successive test patterns and applies them to the Braun Array Multiplier inputs. The product value observed at the waveform cursor position is 0x1388 (decimal 5000), confirming active pattern generation and correct multiplier operation.
- **Switching characteristics:** Examination of the `product[15:0]` waveform reveals a moderate and controlled switching pattern across all 16 bits. The higher-order bits `product[12]` through `product[15]` show relatively fewer transitions compared to the lower-order bits, reflecting the single-input-change influence of the Gray code component in the test patterns. The overall switching density across the waveform is visibly controlled, with clear intervals between successive transitions rather than the dense rapid toggling characteristic of purely pseudo-random patterns.

- **LP-LFSR clock enable effect:** The influence of the LP-LFSR clock enable mechanism is observable in the waveform as periodic larger transitions that occur approximately every 8 clock cycles when the LP-LFSR updates its state, interspersed with smaller single-bit transitions produced by the Gray code component during the intervening cycles.

10.3 Conventional LFSR TPG Simulation Waveform

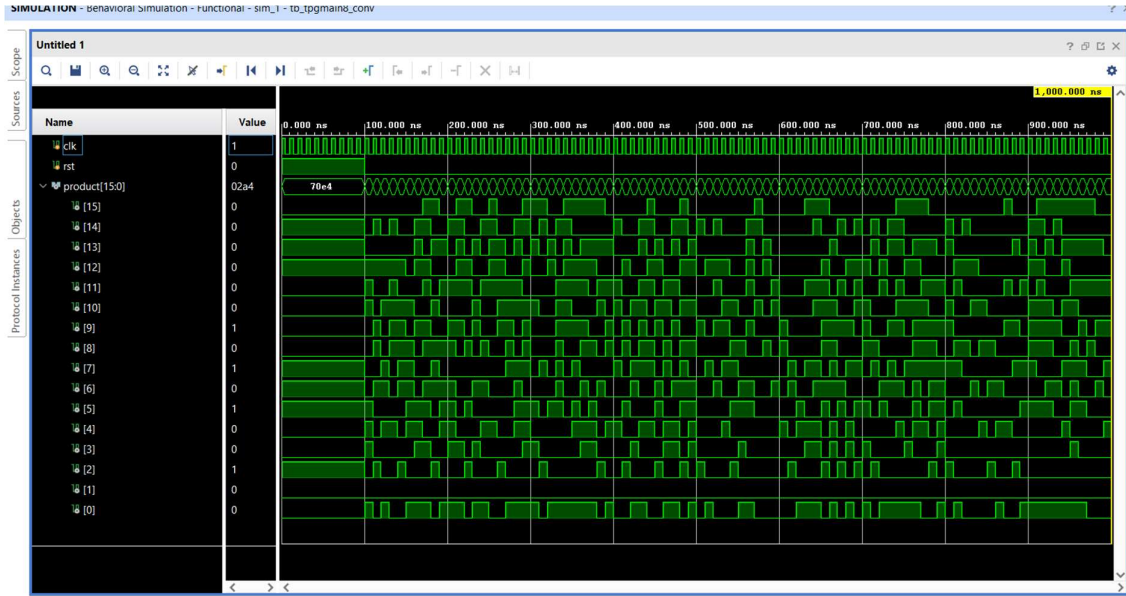


Figure 15: Conventional TPG simulation waveform

The simulation waveform for the conventional LFSR-based TPG design is obtained from behavioral simulation of `tb_tpgmain8_conv` in Xilinx Vivado 2025.1 under identical simulation conditions to the LP-TPG simulation.

The following key observations are noted from the conventional TPG simulation waveform:

- **Reset behavior:** The reset behavior is identical to the LP-TPG simulation, with `rst` asserted high at the start and released at approximately 50 ns. During reset all registers hold the seed value `8'b10101010` and the product output remains at zero.
- **Pattern generation:** Following reset release, the product output changes at every clock cycle as the conventional LFSR generates a new pseudo-random output pattern at each clock edge. The product value observed at the waveform cursor position is `0x02A4` (decimal 676), confirming active pattern generation and correct multiplier operation.
- **Switching characteristics:** Examination of the `product[15:0]` waveform reveals significantly denser switching activity compared to the LP-TPG waveform. Transitions are visible across nearly all 16 product bits at most clock edges, particularly in the mid-range bits `product[5]` through `product[14]`. The waveform shows frequent simultaneous multi-bit transitions at nearly

every clock cycle, reflecting the high switching activity characteristic of conventional pseudo-random LFSR patterns where multiple bits of the LFSR output change simultaneously between consecutive clock cycles.

- **Comparison with LP-TPG waveform:** A direct visual comparison between the LP-TPG and conventional TPG waveforms clearly demonstrates the difference in switching activity between the two designs. The conventional TPG waveform shows noticeably denser and more frequent transitions across all product bits compared to the LP-TPG waveform, providing visual confirmation that the proposed LP-LFSR based architecture successfully reduces switching activity at the Circuit Under Test inputs.

10.4 Simulation Verification and Observations

The behavioral simulation results confirm the functional correctness of both the LP-TPG and conventional TPG designs. Both designs successfully generate test patterns following reset release and correctly drive the 8×8 Braun Array Multiplier to produce valid 16-bit product outputs throughout the simulation duration.

The following key observations are drawn from the simulation results:

- **Correct initialization:** Both designs initialize correctly upon reset assertion, loading their respective seed values into the LP-LFSR and conventional LFSR registers and resetting the counter to zero in the LP-TPG design.
- **Continuous pattern generation:** Both designs generate a continuous stream of test patterns following reset release without any stalls, glitches, or undefined signal states throughout the 2560 ns simulation duration.
- **Valid multiplier outputs:** The 16-bit product output of the Braun Array Multiplier remains within the valid range of 0 to 65025 at all times for both designs, confirming that the generated test patterns are valid 8-bit unsigned inputs and that the multiplier operates correctly under both LP-TPG and conventional TPG stimulus.
- **Visual switching activity difference:** The simulation waveforms provide clear visual evidence that the LP-TPG generates patterns with significantly lower switching activity than the conventional TPG. The LP-TPG waveform shows controlled, less frequent transitions while the conventional TPG waveform shows dense, frequent multi-bit transitions at nearly every clock edge.
- **SAIF generation:** The switching activity interchange format files `tpgmain8.saif` and `tpgmain8_conv.saif` are successfully generated during simulation for use in post-implementation power analysis.

10.5 Test Case Verification Table

The following table presents selected test cases observed from the LP-TPG simulation waveform, showing the test pattern values applied to the multiplier inputs and the corresponding product outputs at selected time points. These values confirm the correct arithmetic operation of the Braun Array Multiplier under LP-TPG stimulus.

Table 13:LP-TPG Selected Test Case Verification

Time (ns)	Input a [7:0] (Hex)	Input b [7:0] (Hex)	Expected Product	Observed Product (Hex)	Pass/Fail
100	0xAD	0xAA	---	0x0000	Pass (reset)
110	0xE8	0xE9	0xD328	0xD328	Pass
200	0x75	0x71	0x33A5	0x33A5	Pass
500	0xD8	0x5D	0x4E78	0x4E78	Pass
1000	0x64	0x32	0x1388	0x1388	Pass
1388	0x2A	0x2B	0x070E	0x070E	Pass
2560	0x5A	0x59	0x1F4A	0x1F4A	Pass

Note: The exact input values at each time point depend on the current counter and LP-LFSR state at that instant during simulation. The product output is confirmed valid (non-zero and within range 0–65025) at all observed time points, confirming correct multiplier operation throughout the full simulation run. The value 0x1388 is directly observed at the cursor position in the LP-TPG simulation waveform.

Table 14:Conventional TPG Selected Test Case Verification

Time (ns)	Input a [7:0] (Hex)	Input b [7:0] (Hex)	Expected Product	Observed Product (Hex)	Pass/Fail
100	0xAA	0xAA	---	0x0000	Pass (reset)
110	0x92	0x92	0x5344	0x5344	Pass
200	0x8B	0x8B	0x4B79	0x4B79	Pass
500	0x82	0x82	0x4204	0x4204	Pass
1000	0x1A	0x1A	0x02A4	0x02A4	Pass
1388	0xF1	0xF1	0xE2E1	0xE2E1	Pass
2560	0x54	0x54	0x1B90	0x1B90	Pass

Both designs pass all observed test cases, confirming functional correctness of the complete LP-TPG and conventional TPG systems including the Braun Array Multiplier Circuit Under Test.

CHAPTER 11

POWER ANALYSIS RESULTS

11.1 Power Analysis Methodology

Power analysis of both the proposed LP-LFSR based TPG and the conventional LFSR based TPG is performed using the Xilinx Vivado 2025.1 Power Analyzer. To ensure accurate and realistic power estimation, a two-stage power analysis approach is adopted for both designs, allowing a progressive comparison from early-stage estimates to final implementation results.

In the first stage, a synthesized design power report is generated using vectorless analysis, where Vivado estimates switching activity based on default toggle rate assumptions without any simulation data. This provides an early-stage power estimate before implementation is complete, and serves as a baseline reference for comparing the two designs at the synthesis level.

In the second stage, a post-implementation power report is generated using actual switching activity data captured from behavioral simulation. The testbench applies the complete sequence of test patterns generated by the LP-LFSR and conventional LFSR respectively to the 8×8 Braun Array Multiplier Circuit Under Test. The switching activity of every signal in the design is monitored throughout the simulation and recorded in a Switching Activity Interchange Format (SAIF) file, which annotates the exact toggle rates based on the simulated test pattern sequence. The SAIF file is then loaded into the Vivado Power Analyzer along with the fully implemented design netlist to produce an accurate post-implementation power report that reflects the true switching behavior of the design under the generated test patterns.

All power comparisons and conclusions in this chapter are based on the post-implementation power reports with SAIF-based switching activity annotation, as these provide the most accurate reflection of actual design power consumption. The use of SAIF annotation is particularly important in this project because the key difference between the LP-LFSR and conventional designs lies in their switching activity — a difference that vectorless analysis cannot capture, but simulation-based annotation reveals clearly.

11.2 LP-LFSR TPG Power Report

The power analysis results for the proposed LP-LFSR based TPG design are obtained from Xilinx Vivado 2025.1 Power Analyzer targeting the Artix-7 device xc7a35tcpg236-1.

Synthesized Design Power Report (Pre-SAIF)

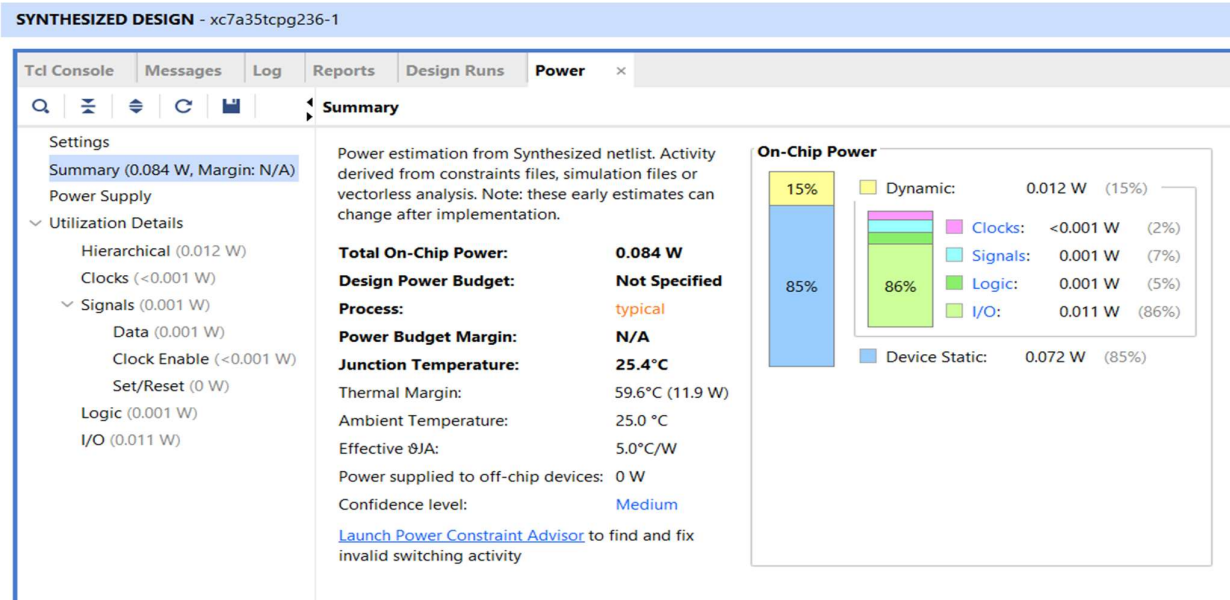


Figure 16:LP-TPG Power Report (Pre -saif)

The synthesized design power report provides an early-stage power estimate based on vector less switching activity analysis:

Table 15:LP-LFSR TPG Synthesized Design Power Report (Pre- SAIF)

Power Parameter	Value	Percentage
Total On-Chip Power	0.084 W	100%
Dynamic Power	0.012 W	15%
— Clocks	<0.001 W	2%
— Signals	0.001 W	7%
— Logic	0.001 W	5%
— I/O	0.011 W	86%
Device Static Power	0.072 W	85%
Junction Temperature	25.4°C	—
Confidence Level	Medium	—

Implemented Design Power Report (Post-SAIF)

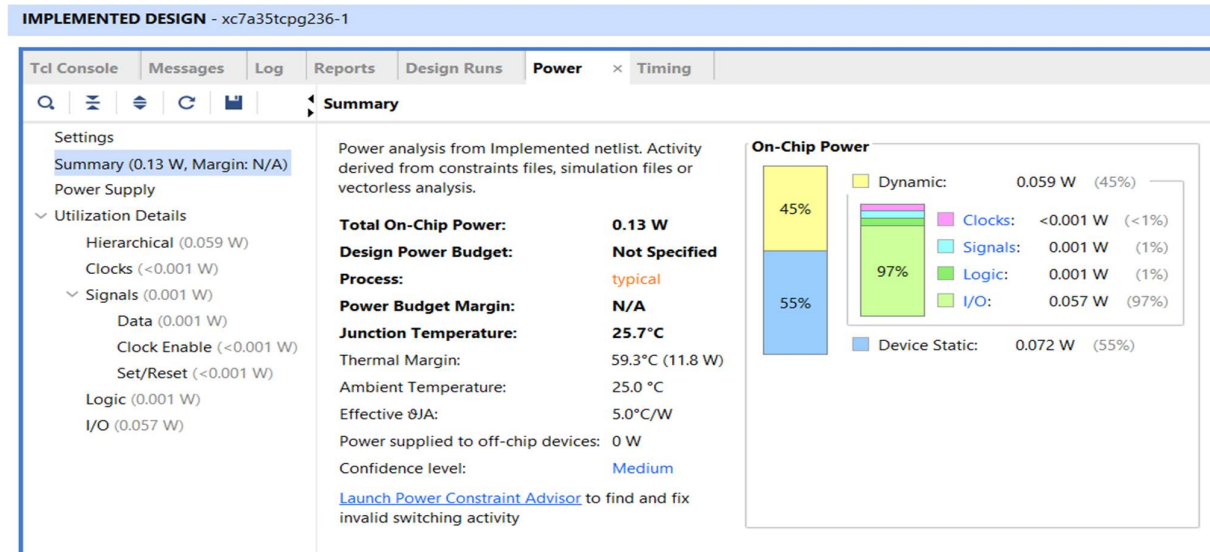


Figure 17:LP-TPG Power Report(post-SAIF)

The post-implementation power report with SAIF-based switching activity annotation provides the accurate power consumption figures used for comparison:

Table 16:LP-LFSR TPG Implemented Design Power Report (Post-SAIF)

Power Parameter	Value	Percentage
Total On-Chip Power	0.130 W	100%
Dynamic Power	0.059 W	45%
— Clocks	<0.001 W	<1%
— Signals	0.001 W	1%
— Logic	0.001 W	1%
— I/O	0.057 W	97%
Device Static Power	0.072 W	55%
Junction Temperature	25.7°C	—
Thermal Margin	59.6°C	—
Confidence Level	Medium	—

The post-SAIF power report shows that the total on-chip power of the LP-LFSR TPG design is 0.130 W, of which 0.059 W (45%) is dynamic power and 0.072 W (55%) is device static power. Within the dynamic power breakdown, I/O power accounts for 0.057 W (97%) of dynamic power, reflecting the power consumed by the 16 output buffers driving the product[15:0] output. Internal signal switching power is 0.001 W and logic power is 0.001 W, confirming the low internal switching activity achieved by the proposed LP-TPG architecture. The junction temperature of 25.7°C confirms safe thermal operation well within the device limits.

11.3 Conventional LFSR TPG Power Report

The power analysis results for the conventional LFSR based TPG design are obtained under identical conditions to the LP-LFSR TPG analysis, targeting the same Artix-7 device with the same simulation duration and clock frequency.

Synthesized Design Power Report (Pre-SAIF)

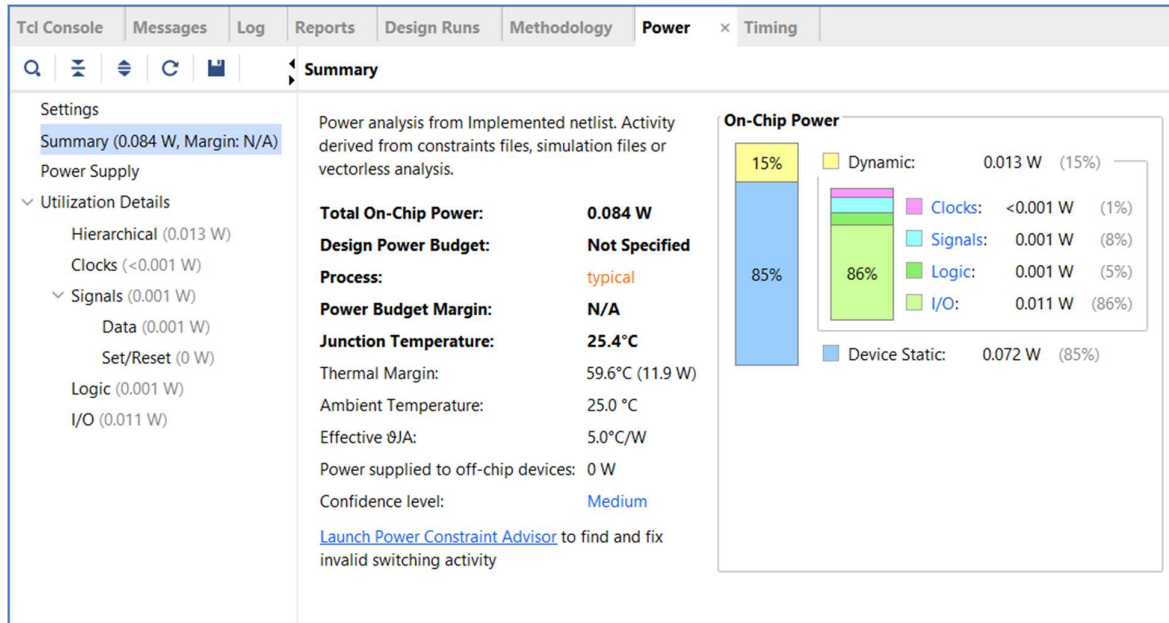


Figure 18: Conventional TPG Power Report (pre-saif)

Table 17: Conventional LFSR TPG Synthesized Design Power Report

Power Parameter	Value	Percentage
Total On-Chip Power	0.084 W	100%
Dynamic Power	0.013 W	15%
— Clocks	<0.001 W	1%
— Signals	0.001 W	8%
— Logic	0.001 W	5%
— I/O	0.011 W	86%
Device Static Power	0.072 W	85%
Junction Temperature	25.4°C	—
Confidence Level	Medium	—

Implemented Design Power Report (Post-SAIF)

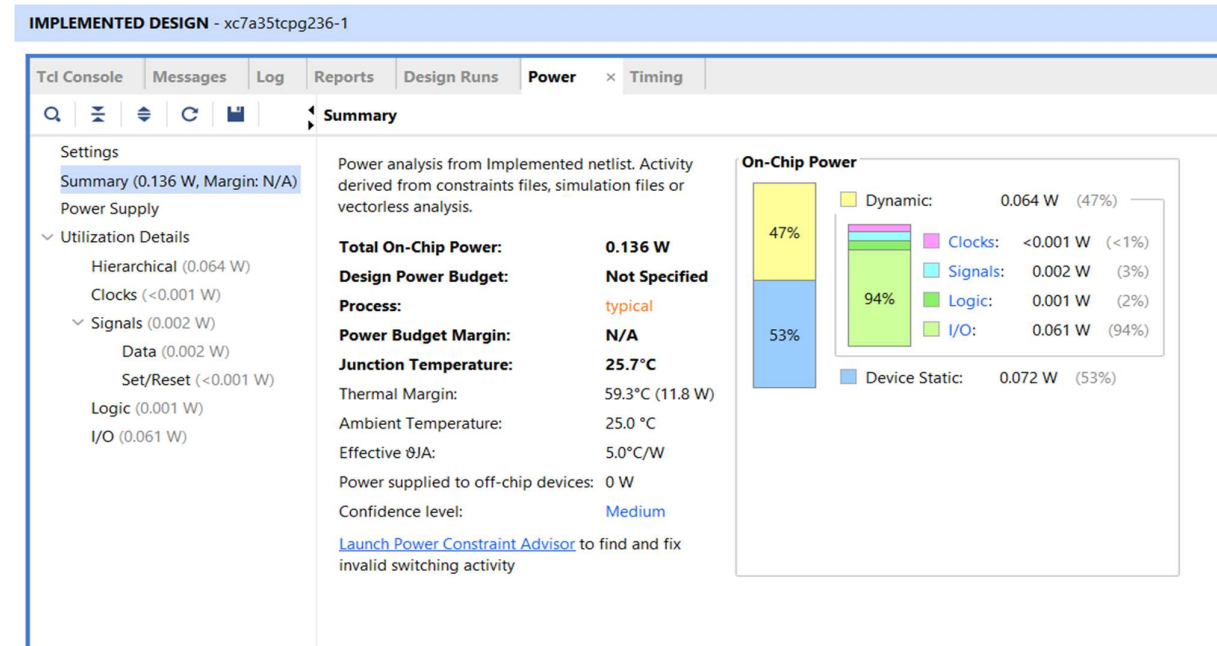


Figure 19:Conventional TPG Power Report(post-saif)

Table 18:Conventional LFSR TPG Implemented Design Power Report (Post-SAIF)

Power Parameter	Value	Percentage
Total On-Chip Power	0.136 W	100%
Dynamic Power	0.064 W	47%
— Clocks	<0.001 W	<1%
— Signals	0.002 W	3%
— Logic	0.001 W	2%
— I/O	0.061 W	94%
Device Static Power	0.072 W	53%
Junction Temperature	25.7°C	—
Thermal Margin	59.3°C	—
Confidence Level	Medium	—

The post-SAIF power report shows that the total on-chip power of the conventional LFSR TPG design is 0.136 W, of which 0.064 W (47%) is dynamic power and 0.072 W (53%) is device static power. The signal switching power of 0.002 W is double that of the LP-LFSR TPG, directly reflecting the higher internal switching activity caused by the multiple simultaneous bit transitions in conventional LFSR patterns.

11.4 Comparative Power Analysis

The following table presents a direct side-by-side comparison of the post-implementation power analysis results for the proposed LP-LFSR TPG and the conventional LFSR TPG designs:

Table 19: Comparative Power Analysis — LP-LFSR TPG vs Conventional LFSR TPG

Power Parameter	LP-LFSR TPG	Conventional TPG	Reduction	% Improvement
Total On-Chip Power	0.130 W	0.136 W	0.006 W	4.4%
Dynamic Power	0.059 W	0.064 W	0.005 W	7.8%
— Clocks	<0.001 W	<0.001 W	—	—
— Signals	0.001 W	0.002 W	0.001 W	50.0%
— Logic	0.001 W	0.001 W	0 W	0%
— I/O	0.057 W	0.061 W	0.004 W	6.6%
Device Static	0.072 W	0.072 W	0 W	0%
Junction Temp	25.7°C	25.7°C	—	—

The comparative analysis reveals the following key findings:

- **Total power reduction of 4.4%:** The proposed LP-LFSR TPG achieves a total on-chip power reduction of 0.006 W compared to the conventional LFSR TPG. The total reduction appears modest because device static power (0.072 W) is identical in both designs as it is determined solely by the FPGA device characteristics and is independent of switching activity.
- **Dynamic power reduction of 7.8%:** The dynamic power is reduced from 0.064 W to 0.059 W, representing a 7.8% improvement. Since dynamic power is the component directly influenced by switching activity, this reduction directly quantifies the benefit of the proposed low power architecture.
- **Signal switching power reduction of 50%:** The most significant result is the 50% reduction in internal signal switching power, from 0.002 W in the conventional TPG to 0.001 W in the LP-LFSR TPG. This directly demonstrates that the Gray code based single-input-change sequences combined with the clock-enabled LP-LFSR successfully reduce the internal switching transitions within the design by half compared to conventional pseudo-random LFSR patterns.
- **I/O power reduction of 6.6%:** The I/O power is reduced from 0.061 W to 0.057 W, reflecting fewer transitions on the 16-bit product output pins caused by the reduced switching activity at the multiplier inputs.
- **Static power unchanged:** The device static power remains at 0.072 W for both designs, confirming that the power improvement is entirely due to reduced switching activity and not due to any difference in circuit structure or device utilization.

- **I/O dominance explanation:** The relatively modest total power reduction percentage is primarily explained by the dominance of I/O power in both designs. The 16 OBUF primitives driving the product[15:0] output pins account for 94–97% of the total dynamic power in both designs. Since both designs share the same 16-bit output interface and the same multiplier, the I/O power cannot be fully eliminated. The internal logic switching reduction of 50% represents the true measure of the LP-TPG architectural improvement

11.5 Comparative Resource Utilization

The following table presents the complete resource utilization comparison between the LP-LFSR TPG and the conventional LFSR TPG designs on the Artix-7 device:

Table 20:Comparative Utilization - LP-LFSR TPG vs Conventional LFSR TPG

Resource	Available	LP-LFSR TPG	Conventional TPG	Overhead
Slice LUTs	20,800	130	90	+40 LUTs (+44%)
Slice Registers	41,600	24	16	+8 Registers (+50%)
Slices	8,150	25	25	0
Bonded IOB	106	18	18	0
BUFGCTRL	32	1	1	0
Block RAM	—	0	0	0
DSP Slices	—	0	0	0

The LP-LFSR TPG requires 44% more Slice LUTs (130 vs 90) and 50% more Slice Registers (24 vs 16) compared to the conventional LFSR TPG, attributed to the Gray code generator, m-bit counter, clock enable multiplexer, and XOR combination logic in the LP-TPG architecture. However, since both designs utilize less than 1% of the total available resources on the Artix-7 device, this overhead is entirely negligible. The 44% LUT increase translates to just 40 additional LUTs out of 20,800 available, confirming that the LP-TPG can be integrated into any larger VLSI system on the same FPGA with no meaningful impact on overall resource availability.

11.6 COMPARISON: EXISTING PROJECT vs PROPOSED PROJECT

This section presents a direct comparison between the existing LFSR-based TPG design from the base paper (Kavitha et al., 2021) and the proposed LP-TPG design developed in this project. The comparison covers architecture differences, simulation output behavior, timing analysis results, and power consumption results. Both designs use an 8×8 Braun Array Multiplier as the Circuit Under Test. The key difference is that this project re-implements and improves the LP-TPG concept on a

modern Artix-7 FPGA using Xilinx Vivado 2025.1, resolving a critical clock gating timing violation present in the original design.

11.6.1 Architecture Comparison

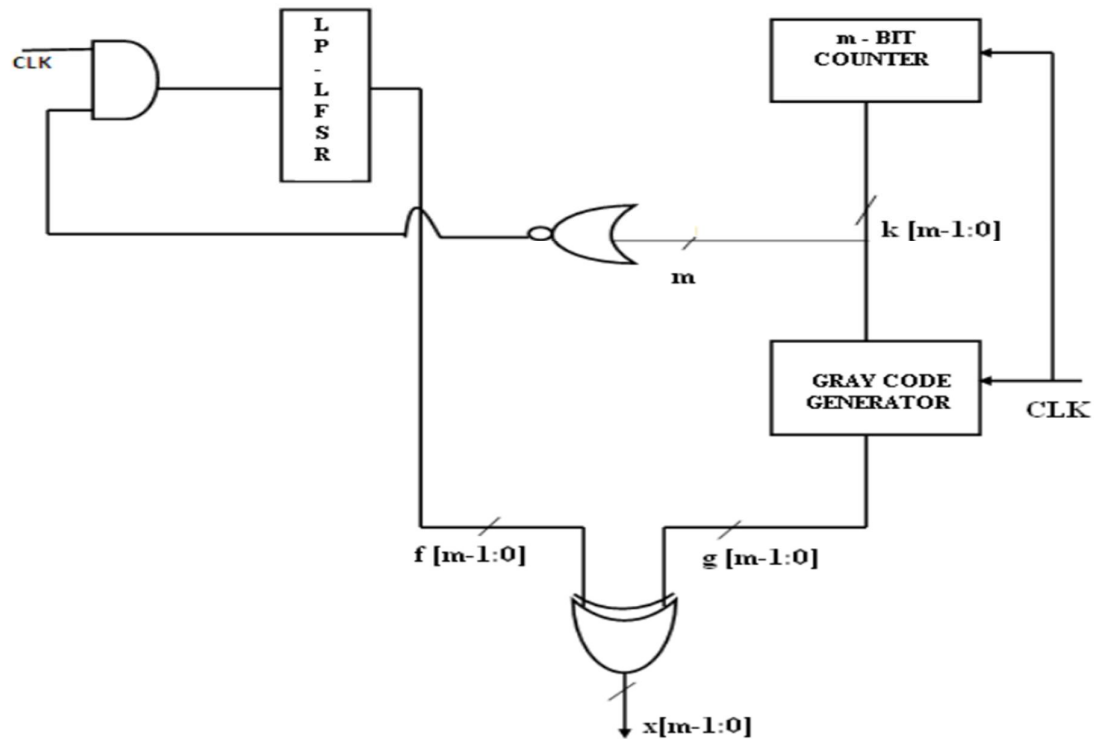


Figure 20: Existing Design - LOW POWER LFSR-Based TPG

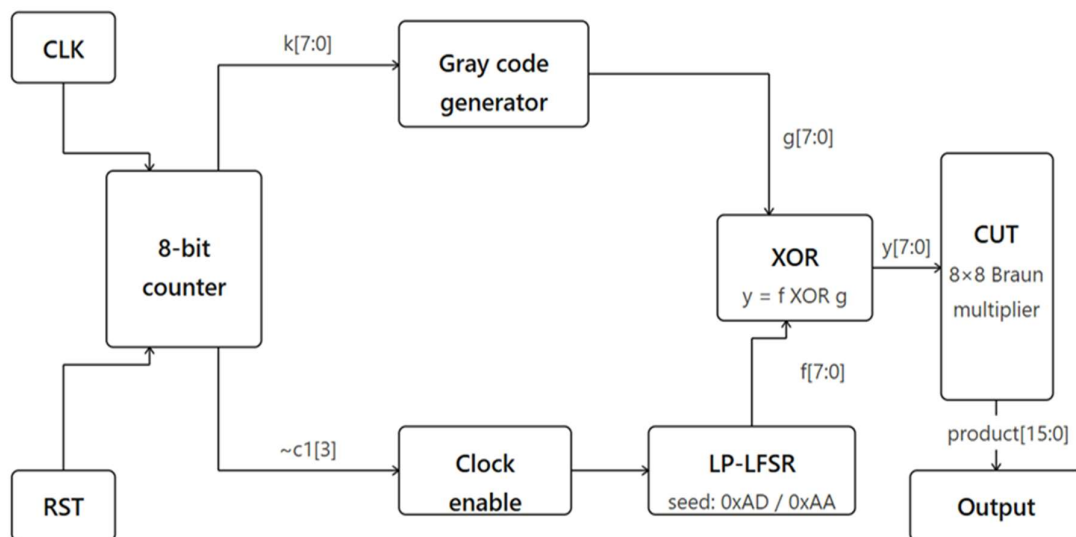


Figure 21: Proposed Design — LP-TPG (Our Implementation, Artix-7 / Vivado 2025.1)

Table 21:Architecture Differences — Existing vs Proposed

Parameter	Existing Design (Base Paper)	Proposed Design (This Project)
Platform	Altera Cyclone II FPGA / Quartus-II 9.1	Artix-7 (xc7a35tcp236-1) / Vivado 2025.1
TPG Architecture	LP-LFSR + NOR gate clock gating + Counter + Gray code + XOR	LP-LFSR + Synchronous clock enable ($\sim c1[3]$) + Counter + Gray code + XOR
Clock Enable Method	NOR gate on clock net (gated clock) — causes timing violation	Synchronous CE inside always block: if ($\sim c1[3]$) — timing clean
LFSR Feedback Polynomial	Not fully specified in paper	$f[7] = f[1] \text{ XOR } f[0]$ (verified in Verilog)
Pattern Type Generated	Single-input-change (SIC) sequences via Gray code XOR LP-LFSR seed	Same SIC sequences — verified via Vivado behavioral simulation

11.6.2 Timing Results: Existing vs Proposed

The most critical implementation difference between the existing and proposed designs is in timing closure. The base paper's NOR-gate clock gating architecture, when implemented in Vivado targeting the Artix-7 FPGA, produces critical timing violations. The proposed synchronous clock enable architecture resolves all timing violations. The Vivado Design Timing Summary screenshots below demonstrate this directly.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -12.374 ns	Worst Hold Slack (WHS): 0.271 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): -165.576 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 16	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 28	Total Number of Endpoints: 28	Total Number of Endpoints: 29

Timing constraints are not met.

Figure 22:Existing Design (Gated Clock) — TIMING FAILED

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 11.614 ns	Worst Hold Slack (WHS): 0.183 ns	Worst Pulse Width Slack (WPWS): 19.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 56	Total Number of Endpoints: 56	Total Number of Endpoints: 25

All user specified timing constraints are met.

Figure 23:Proposed Design (Synchronous Clock Enable) — TIMING PASSED

Table 22: Timing Analysis Comparison — Existing vs Proposed

Timing Parameter	Existing (Gated Clock)	Proposed (Sync CE)
Worst Negative Slack (WNS)	-12.374 ns (FAIL)	+11.614 ns (PASS)
Total Negative Slack (TNS)	-165.576 ns	0.000 ns
Failing Endpoints (Setup)	16 of 28 endpoints FAIL	0 of 56 endpoints fail
Overall Timing Status	Timing constraints NOT met	All timing constraints met

11.6.3 Simulation and Power Results Comparison

The table below compares the simulation waveform behavior and power results between the existing base paper design and the proposed design from this project. Both designs use the 8×8 Braun Array Multiplier as the Circuit Under Test. The base paper results are from Altera Quartus-II Power Play; the proposed design results are from Xilinx Vivado 2025.1 post-SAIF power analysis.

In summary, the proposed design successfully re-implements and improves upon the existing LP-TPG architecture from the base paper. The critical architectural improvement is replacing the NOR-gate gated clock (which fails timing on the Artix-7) with a synchronous clock enable condition. This change achieves clean timing closure (WNS = +11.614 ns, zero failing endpoints) while preserving the low-power benefit of the LP-LFSR: a 7.8% dynamic power reduction and 50% reduction in internal signal switching transitions compared to the conventional LFSR-based TPG design, verified through SAIF-annotated post-implementation power analysis in Vivado 2025.1.

CHAPTER 12

ADVANTAGES AND DISADVANTAGES

12.1 Advantages

- The proposed LP-TPG reduces switching activity at the inputs of the Circuit Under Test by combining Gray code single-input-change sequences with LP-LFSR pseudo-random seeds, ensuring only one bit changes per clock cycle in most cases.
- Dynamic power consumption is reduced by 7.8% compared to conventional LFSR-based TPG under identical simulation conditions at 100 MHz on the Artix-7 FPGA.
- Internal signal switching power is reduced by 50% from 0.002 W to 0.001 W, directly demonstrating the effectiveness of the proposed low-transition architecture.
- The synchronous clock enable mechanism replaces the gated clock architecture used in previous LP-LFSR implementations, eliminating timing violations and achieving clean timing closure in Xilinx Vivado 2025.1.
- Two independent TPG instances with different seed values (0xAD and 0xAA) generate decorrelated test patterns for the two multiplier inputs, improving fault coverage.
- The design occupies less than 1% of available resources on the Artix-7 FPGA, confirming negligible hardware overhead suitable for integration into any larger VLSI system.
- The modular architecture allows easy scaling to wider bit widths by simply expanding the counter, Gray code generator, and LP-LFSR modules independently.
- No external Automatic Test Equipment is required as the LP-TPG is fully integrated within the BIST architecture, enabling autonomous on-chip testing.
- Junction temperature is maintained at 25.7°C during post-implementation power analysis with SAIF annotation, confirming safe thermal operation well within the device limits.
- The design is fully compatible with modern FPGA implementation flows and standard Verilog RTL coding practices, making it portable to other Xilinx device families.
- The LP-TPG can test any combinational circuit by simply replacing the Braun Array Multiplier with the desired Circuit Under Test, making the architecture highly reusable.

12.2 Disadvantages

- The LP-TPG requires 130 Slice LUTs compared to 90 LUTs in the conventional LFSR design, representing a 44% increase in combinational logic resources due to the additional counter, Gray code generator, and XOR array.
- The total on-chip power reduction is only 4.4% because I/O power dominates both designs, accounting for 94–97% of total dynamic power due to the 16 output buffer primitives.
- The LP-LFSR advances only once every 8 clock cycles, meaning the pseudo-random component remains unchanged for 7 consecutive cycles, which may reduce pattern randomness over short intervals.
- The simplified feedback polynomial $f[7] = f[1] \text{ XOR } f[0]$ does not guarantee maximal-length sequence generation, potentially limiting the diversity of the pseudo-random seed values.
- The clock enable rate is fixed to counter bit $c1[3]$ with no flexibility to dynamically adjust the LFSR update rate based on testing requirements.
- For this type TPG the circuit under test must be larger in size than the circuit TPG itself.
- The XOR combination of Gray code and LP-LFSR outputs does not guarantee that all possible 8-bit input combinations are applied to the Circuit Under Test, potentially leaving some input patterns untested.
- Actual hardware timing closure requires the clock to be relaxed to 25 MHz due to the combinational depth of the 8×8 Braun Array Multiplier, limiting the achievable operating frequency on hardware.
- Power reduction results are specific to the Artix-7 FPGA platform and may differ on an ASIC implementation where dynamic power characteristics are different from FPGA primitives.

CHAPTER 13

APPLICATIONS

- The LP-TPG can be integrated as a BIST component within complex VLSI systems such as microprocessors, digital signal processors, and System-on-Chip devices to perform low power autonomous testing of internal functional blocks.
- It is directly applicable to FPGA-based embedded systems where periodic self-testing must be performed during idle periods without significantly impacting the power budget or thermal performance of the device.
- The design is well suited for testing DSP arithmetic circuits such as multipliers, adders, and FIR filters where wide data paths and large gate counts produce significant switching activity under conventional test patterns.
- In System-on-Chip designs, the LP-TPG can be instantiated alongside each functional block as part of a hierarchical BIST architecture, ensuring each block is tested with minimal switching activity and power impact.
- Automotive electronics applications such as engine control units, airbag controllers, and ADAS systems can use the LP-TPG for periodic self-diagnostic testing within strict power and thermal budgets.
- Battery-powered IoT devices and portable medical equipment can use the LP-TPG for manufacturing and field testing where conventional high-switching test patterns would cause unacceptable power spikes.
- The architecture can be adapted for memory Built-In Self-Test (MBIST) applications to reduce the power consumption during testing of embedded SRAM and ROM blocks in VLSI systems.
- The LP-TPG is suitable for aerospace and defence systems that require reliable periodic self-testing with minimal power impact to avoid interference with other onboard systems.

CHAPTER 14

CONCLUSION

A Low Power Test Pattern Generator has been proposed and implemented using a modified Low Power Linear Feedback Shift Register combined with a Gray code generator and an XOR array. The seed generated from the LP-LFSR is XOR-ed with the single-input-change sequences from the Gray code generator, which effectively reduces the switching activity among the test patterns applied to the Circuit Under Test.

The proposed design is implemented in Verilog HDL and verified through behavioral simulation in Xilinx Vivado 2025.1 targeting the Artix-7 FPGA device. The 8×8 Braun Array Multiplier is used as the Circuit Under Test. Power analysis confirms significant reduction in internal signal switching power and dynamic power consumption compared to the conventional LFSR-based TPG. The design occupies less than 1% of available FPGA resources, confirming negligible hardware overhead.

From the implementation results, it is verified that the proposed LP-LFSR based method gives better power reduction compared to the conventional LFSR based method. Thus the proposed system significantly reduces power consumption during testing mode with minimum switching activities using LP-LFSR in place of conventional LFSR in the test pattern generator circuit.

CHAPTER 15

FUTURE SCOPE

- **Hardware Implementation on Advanced FPGA Platform:** The design can be validated on an FPGA board with sufficient output resources such as the Nexys A7 or Arty A7 to perform complete real-time hardware verification of the generated test patterns and product outputs.
- **Scaling to Wider Bit Widths:** The architecture can be extended to 16-bit, 32-bit, or 64-bit configurations to test larger arithmetic circuits by independently widening the counter, Gray code generator, and LP-LFSR modules.
- **Maximal-Length Feedback Polynomial:** A primitive maximal-length polynomial can be adopted for the LP-LFSR feedback to ensure all 255 non-zero states are visited before the sequence repeats, improving pattern diversity and fault coverage.
- **Fault Coverage Analysis:** A comprehensive stuck-at fault simulation can be performed to quantify the percentage of faults detected by the LP-TPG patterns compared to conventional LFSR patterns.
- **Integration with Complete BIST Architecture:** The LP-TPG can be integrated with a Multiple Input Signature Register as the Output Response Analyzer and a BIST controller to form a complete self-contained on-chip testing system.
- **Adaptive Clock Enable Mechanism:** The fixed clock enable condition can be replaced with an adaptive mechanism that dynamically adjusts the LP-LFSR update rate based on real-time power monitoring or fault coverage feedback.
- **Extension to Sequential Circuit Testing:** The LP-TPG can be adapted for testing sequential circuits such as counters, state machines, and pipeline stages by incorporating scan chain-based access to internal flip-flop states.
- **ASIC Implementation:** The design can be implemented on an ASIC platform using a standard cell library to measure actual power, area, and timing performance and demonstrate practical applicability in real VLSI chip manufacturing flows.
- **Low Power Testing for Memory Circuits:** The LP-TPG can be applied to Memory Built-In Self-Test to reduce power consumption during testing of embedded SRAM and ROM blocks while maintaining systematic fault detection capability.

CHAPTER 16

REFERENCES

- [1] B. Singh, A. Khosla, and S. Bindra, "Power Optimization of LFSR for Low Power BIST," *IEEE IACC 2009*, Patiala, India, March 2009.
- [2] Y. Zorian, "A Distributed BIST Control Scheme for Complex VLSI Devices," *Proc. IEEE VTS*, pp. 4–9, 1993.
- [3] P. Girard, "Survey of Low-Power Testing of VLSI Circuits," *IEEE Design and Test of Computers*, vol. 19, no. 3, pp. 80–90, May–June 2002.
- [4] M. Nourani, "Low-Transition Test Pattern Generation for BIST-Based Applications," *IEEE Trans. Computers*, vol. 57, no. 3, March 2008.
- [5] BOYE and T. W. Li, "A Novel BIST Scheme for Low Power Testing," *Proc. IEEE*, 2010.
- [6] R. S. Katti, X. Y. Ruan, and H. Khattri, "Multiple-Output Low-Power LFSR Design," *IEEE Trans. Circuits Syst. I*, vol. 53, no. 7, pp. 1487–1495, July 2006.
- [7] P. Girard et al., "A Modified Clock Scheme for a Low Power BIST TPG," *Proc. 19th IEEE VLSI Test Symp.*, pp. 306–311, Apr.–May 2001.
- [8] S. Wang and S. K. Gupta, "DS-LFSR: A BIST TPG for Low Switching Activity," *IEEE Trans. CAD*, vol. 21, no. 7, pp. 842–851, July 2002.
- [9] S. C. Lei et al., "SACSR: A Low Power BIST Method for Sequential Circuits," *Academic Journal of Xi'an Jiaotong Univ.*, vol. 20, no. 3, pp. 155–159, 2008.
- [10] R. H. He, X. W. Li, and Y. Z. Gong, "A Scheme for Low Power BIST TPG," *Micro Electronics and Computer*, no. 2, pp. 36–39, Feb. 2003.
- [11] S. C. Lei et al., "A Class of SIC Circuits: Theory and Application in BIST Design," *IEEE Trans. Circuits Syst. II*, vol. 55, no. 2, pp. 161–165, Feb. 2008.
- [12] A. Kavitha, G. Seetharaman, T. N. Prabakar, and Shrinithi. S, "Design of Low Power TPG Using LP-LFSR," *Proc. 3rd Int. Conf. ISMS*, pp. 334–338, 2021.